

**M A S A R Y K O V A
U N I V E R Z I T A**

FAKULTA INFORMATIKY

**Srovnání zpracování logů z
buildování RPM balíčků pomocí
více LLM**

Diplomová práce

BC. DANIEL NĚMEC

Brno, podzim 2025

**M A S A R Y K O V A
U N I V E R Z I T A**

FAKULTA INFORMATIKY

**Srovnání zpracování logů z
buildování RPM balíčků pomocí
více LLM**

Diplomová práce

BC. DANIEL NĚMEC

Vedoucí práce: Mgr. Marek Grác, Ph.D.

Katedra strojového učení a zpracování dat

Brno, podzim 2025



Prohlášení

Prohlašuji, že tato diplomová práce je mým původním autorským dílem, které jsem vypracoval samostatně. Všechny zdroje, prameny a literaturu, které jsem při vypracování používal nebo z nich čerpal, v práci řádně cituji s uvedením úplného odkazu na příslušný zdroj.

Bc. Daniel Němec

Vedoucí práce: Mgr. Marek Grác, Ph.D.

Poděkování

Tímto děkuji za vedení a pomoc při vypracování diplomové práce vedoucímu práce panu Mgr. Marku Grácovi, Ph.D. Dále také děkuji panu Jiřímu Podivínovi za technické a odborné konzultace.

Shrnutí

Práce se zabývá komparativním vyhodnocením velkých jazykových modelů (LLM) při analýze neúspěšných build logů z procesu sestavování RPM balíčků. Cílem je objektivně posoudit jejich schopnost identifikovat příčinu selhání, formulovat kvalitní a fakticky správné vysvětlení a navrhnout vhodné řešení. Součástí práce je definice evaluační metodiky kombinující referenční (ROUGE-L, BERTScore) i bez-referenční přístupy včetně hodnoticího rámce G-Eval využívajícího princip LLM-as-a-judge. Pro měření výkonnosti jsou sledovány metriky přesnosti lokalizace chyby, kvality popisu, faktuální konzistence, bezpečnosti generovaných návrhů, latence a nákladů. Experimenty jsou provedeny nad sadou 60 reálných selhaných RPM buildů. Součástí práce je návrh a implementace evaluační pipeline nad frameworkem DeepEval, který umožňuje plně reprodukovatelné vyhodnocení různých LLM a jejich konfigurací. Výsledky ukazují výrazné rozdíly mezi jednotlivými modely jak v kvalitě analýzy, tak v provozních charakteristikách, a poskytují systematické doporučení pro jejich integraci do nástroje Log Detective.

Klíčová slova

LLM, RPM, Log Detective, evaluace, benchmark, automatizovaná analýza logů

Obsah

Úvod	1
1 Log Detective	3
1.1 Funkcionalita	3
1.2 Problém	4
2 RPM balíček	7
2.1 Sestavování RPM balíčků	7
3 LLM	11
3.1 Teplota modelu	11
3.2 Modely	12
4 Evaluation	16
4.1 Dataset	16
4.2 Metody měření	17
4.2.1 Referenční metriky	18
4.2.2 Bezreferenční metriky	19
4.3 Měřené metriky	22
5 Frameworky pro vyhodnocování a automatizaci	26
5.1 Požadavky na evaluační framework	26
5.2 Platformy	27
5.2.1 DeepEval	27
5.2.2 Arize Phoenix	27
5.2.3 Langfuse	27
5.3 Výběr frameworku	27
6 Výsledky	30
6.1 Výsledky podle metrik	30
6.2 Multi-criteria decision analysis	45
6.3 Vliv parametru temperature	48
Závěr a zhodnocení	49
Bibliografie	52

Seznam tabulek

3.1	Přehled modelů pro vyhodnocování	15
4.1	Přehled hodnocených metrik a jejich metod měření	25
6.1	Průměrné hodnoty přesnosti lokalizace chyby	32
6.2	Průměrné hodnoty kvality popisu chyby	35
6.3	Průměrné hodnoty faktické konzistence	38
6.4	Cena za 1 milion tokenů. Ceny platné ke dni 21.11.2025 .	43
6.5	Váhy pro vícekritériální rozhodovací analýzu (MCDA) . .	45

Seznam obrázků

6.1	Distribuce metriky přesnosti lokalizace chyby pro všechny modely	31
6.2	Distribuce metriky kvality popisu chyby pro všechny modely	33
6.3	Vztah mezi přesností lokalizace a kvalitou popisu chyby .	36
6.4	Distribuce hodnot faktické konzistence pro všechny modely	37
6.5	Distribuce hodnot bezpečnosti pro všechny modely	39
6.6	Průměrná latence jednotlivých modelů	41
6.7	Průměrné náklady jednotlivých modelů	42
6.8	Kompromisy mezi výkonnostními a provozními metrikami	44
6.9	MCDA pořadí modelů	46
6.10	Vliv teploty na jednotlivé měřené metriky	49

Úvod

Budování softwarových balíčků představuje zásadní součást údržby moderních open-source linuxových systémů. Jedním z nejrozšířenějších formátů je RPM (Red Hat Package Manager), používaný v distribucích, jako jsou Fedora, Red Hat Enterprise Linux (RHEL) či CentOS [31]. Při sestavování RPM balíčku vzniká podrobný záznam o průběhu kompilace. Tento záznam obsahuje informace o všech krocích procesu, od stahování závislostí a přípravy zdrojového kódu až po kompilaci a tvorbu výsledného balíčku. Tyto logy mají klíčový význam, neboť vývojářům umožňují identifikovat a odstraňovat chyby, které během sestavení vznikly.

RPM build logy však bývají velmi rozsáhlé a komplexní. Často obsahují tisíce řádků s výstupy překladače, varováními, kontrolami závislostí či chybovými hlášeními. Skutečná příčina chyby bývá mnohdy skryta hluboko v záznamu. I zkušení vývojáři mohou strávit značné množství času jejím hledáním a identifikací, pro začátečníky nebo automatizované systémy je tato úloha ještě obtížnější. Záznamy se rovněž liší podle projektu, což komplikuje tvorbu univerzálního nástroje, který by fungoval napříč různými případy.

Rychlá a přesná detekce chyb je důležitá pro stabilitu i rychlost vývoje softwaru. Pokud lze proces detekce chyb zefektivnit a zpřesnit, mohou se vývojáři více soustředit na řešení samotného problému namísto procházení rozsáhlých logů. To může ušetřit čas, snížit náklady a zprehlednit celý proces sestavování softwaru.

V posledních letech se velké jazykové modely (Large Language Models, LLMs), jako GPT, Gemini, LLaMA nebo Mistral, ukázaly jako výrazně schopné při práci s obtížnými, nestrukturovanými texty. LLM dokáží analyzovat kontext, sumarizovat informace a zvýraznit klíčové části textu. To z nich činí perspektivní technologie pro automatizovanou analýzu RPM build logů či podporu vývojářů při jejich vyhodnocování.

V návaznosti na tento trend vznikly open-source nástroje, jako je Log Detective. Cílem nástroje Log Detective je ulehčit vývojářům práci s logy neúspěšných sestavení a integrovat jejich analýzu do automatizovaných procesů. Pokud jsou LLMs propojeny s nástrojem typu Log Detective, lze vybudovat systém, který nejen čte a sumarizuje logy,

ale současně identifikuje pravděpodobnou příčinu chyby a navrhuje řešení.

Cílem této práce je porovnat několik velkých jazykových modelů a zjistit, jak dobře dokáží analyzovat RPM build logy, v rámci nástroje Log Detective. Porovnání se zaměřuje na několik klíčových aspektů:

- schopnost modelu nalézt chybové místo
- kvalitu a přesnost vysvětlení logu
- užitečnost navrženého řešení
- bezpečnost a spolehlivost výstupu
- rychlost a náklady na analýzu

Pro hodnocení je využita sada metrik zahrnujících referenční i bezreferenční metody, doplněná o testování bezpečnosti a měření provozních charakteristik modelů. Na základě těchto dat je provedeno více-kritériální porovnání jednotlivých LLM a identifikace modelů, které poskytují nejvhodnější poměr mezi přesností, konzistencí, spolehlivostí a provozní efektivitou.

Cílem práce je tedy poskytnout objektivní a reprodukovatelné srovnání aktuálně dostupných jazykových modelů v úloze analýzy RPM build logů a vytvořit podklad pro jejich případnou integraci do existujících nástrojů používaných při správě balíčků a automatizaci sestavování v linuxových distribucích.

1 Log Detective

Log Detective je open-source projekt, který v současné době spravují Tomáš Tomeček a Jiří Podivín. Jedná se o nástroj využívající umělou inteligenci k analýze build logů RPM balíčků, identifikaci neúspěšných sestavení, vyhodnocení příčin selhání a návrhu možných řešení problému. Motivací projektu je zjednodušit a urychlit práci s rozsáhlými build logy, které mohou mít tisíce řádků a kde výskyt řetězce „ERROR“ nemusí nutně znamenat skutečnou chybu. Současně se projekt snaží omezit potřebu detailní znalosti RPM balíčkovacího ekosystému, jelikož prohlížení celých logů často vyžaduje určitou míru expertízy [21].

Aktuální vývoj se zaměřuje na shromažďování neúspěšných build logů anotovaných relevantními částmi výstupu a popisem způsobu opravy. Očekává se, že k trénování specializovaných modelů je zapotřebí velké množství takových logů, ideálně v řádech tisíců.

1.1 Funkcionalita

Log Detective využívá knihovnu pro extrakci logových šablon nazývanou Drain3. Drain3 je open-source nástroj pro těžbu logů v reálném čase založený na algoritmu Drain. Automaticky seskupuje příchozí logové zprávy do strukturovaných šablon pomocí pevně stanovené hloubky stromu, což umožňuje efektivní a rychlou identifikaci vzorů v logových datech. Díky nízkým nárokům na paměť a minimalistickému návrhu je Drain3 vhodný pro oblasti, jako jsou pozorování, měření a detekce anomálií[22]. Projekt poskytuje dvě rozhraní:

1. Příkazovou řádku (CLI) pro správce a balíčkáře pracující lokálně.
2. Službu postavenou na FastAPI, která se integruje do systémů kontinuální integrace.

V režimu CLI uživatel spouští příkaz `logdetective <cesta-nebo-url>` s volitelnými parametry, které umožňují výběr lokálního nebo Hugging Face GGUF modelu, úpravu hloubky shlukování Drain3

nebo použití vlastního promptu. Nástroj stáhne nebo načte log, rozdělí jej na části pomocí Drain3, zachová čísla řádků a zpráhledněné segmenty následně předá modelu přes knihovnu llama.cpp. Výstup v terminálu obsahuje jednotlivé části logu doplněné vysvětlující větou a na konci také mediánovou hodnotu „důvěry“, která uživateli napovídá, nakolik se lze výstupem řídit. Současná verze Log Detective používá Mistral 7b instruct jako vyhodnovací velký jazykový model.

Režim postavený na FastAPI, využívá stejnou pipeline, ale umožňuje integraci přes HTTP rozhraní. FastAPI poskytuje například endpointy /analyze a /analyze/staged, které přijímají JSON obsahující URL k logu. Ve „staged“ režimu jsou jednotlivé části logu analyzovány samostatně a volitelně lze využít filtraci n nejrelevantnějších segmentů. GitLab webhooks mohou posílat události z neúspěšných úloh build_rpm na endpoint /webhook/gitlab/job_events. Nástroj následně stáhne artefakty, extrahuje soubor task_failed.log, provede analýzu, uloží výsledek do databáze PostgreSQL a připojí shrnutí v Markdownu přímo do merge requestu.

Analogický postup se používá i při integraci s Koji. Uživatel poskytne instanci a ID úlohy, systém stáhne log pomocí XML-RPC, zařadí analýzu do fronty a odešle výsledek zpět, pokud je povolen callback. Všechny odpovědi služby zahrnují analyzované segmenty, jejich vysvětlení a skóre důvěry. Součástí služby jsou také metrické endpointy generující grafy ve formátu SVG, které zobrazují počet požadavků, latenci nebo reakce uživatelů.

1.2 Problém

Log Detective kombinuje těžbu logových šablon s analýzu pomocí velkého jazykového modelu. Cílem je automatizovat identifikaci a vysvětlení selhání při sestavování RPM balíčků. Těžba šablon redukuje množství šumu a zjednodušuje strukturu logu, nicméně celková kvalita systému výrazně závisí na schopnostech použitého jazykového modelu.

Hlavní výzvou je výběr takového modelu, který dokáže přesně lokalizovat relevantní část logu obsahující skutečnou chybu a následně poskytnout konzistentní, fakticky správné a bezpečné vysvětlení. Jednotlivé modely se výrazně liší v kvalitě usuzování, míře halucinací,

rychlosti i provozních nákladech, a jejich dopad na celkovou spolehlivost systému je proto zásadní.

Ačkoli v posledních letech vznikly první systematické benchmarky hodnotící výkonnost velkých jazykových modelů v úlohách souvisejících s logy, jejich zaměření se významně liší od požadavků nástrojů typu LogDetective.

LogEval představuje rozsáhlý benchmark pokrývající čtyři klíčové oblasti analýzy logů: log parsing, detekci anomálií, diagnostiku chyb a sumarizaci. Porovnává desítky modelů na více než čtyřech tisících logů pocházejících z produkčních systémů. Cílem LogEval je ověřit schopnost modelů pracovat s již existujícími logy, nikoliv však lokalizovat konkrétní příčiny selhání v rozsáhlých sestavovacích výstupech či generovat vysvětlení použitelná v kontextu CI/CD [4].

Doplňující perspektivu nabízí Defects4Log, který se zaměřuje na odlišnou úroveň problému, konkrétně na detekci defektů v samotných logovacích instrukcích. Autoři definují taxonomii sedmi typů chyb v log statements a hodnotí schopnost jazykových modelů tyto defekty identifikovat a odůvodnit. Výsledky ukazují, že současné modely mají obtíže zejména v oblasti hlubší sémantické analýzy kódu. Benchmark se však nezabývá analýzou komplexních runtime logů ani generováním jejich vysvětlení [34].

Obě studie představují významné příspěvky k hodnocení jazykových modelů v úlohách souvisejících s logy, nicméně žádná z nich neřeší specifický problém lokalizace chyb v RPM build logách ani kvalitu následného vysvětlení, což je pro nástroje typu LogDetective klíčové. V současné době tak chybí benchmark zaměřený přímo na tento scénář. Tato diplomová práce tuto mezeru zaplňuje prostřednictvím komparativního vyhodnocení moderních jazykových modelů z hlediska přesnosti identifikace chyby a kvality generovaného vysvětlení.

Pro účely benchmarku je využíván výchozí endpoint /analyze, jelikož práce nehodnotí extrémně velké logy. Přitom je použit také výchozí systémový prompt, který definuje očekávané chování modelu, a šablona uživatelského promptu, do níž jsou vkládány segmenty extrahované Drain3. Benchmark je pro každý model proveden pokaždé s jinou hodnotou parametru temperature (0.0, 0.4, 0.8), aby bylo možné posoudit determinismus modelu a variabilitu jeho odpovědí. Systém-

mový prompt a uživatelský prompt pro LogDetective v čase provedení benchmarku vypadají následovně:

Systémový prompt

You are a highly capable large language model based expert system specialized in packaging and delivery of software using RPM (RPM Package Manager). Your purpose is to diagnose RPM build failures, identifying root causes and proposing solutions if possible.

You are truthful, concise, and helpful. You never speculate about package being built or fabricate information. If you do not know the answer, you acknowledge the fact and end your response. Your responses must be as short as possible.

Uživatelský prompt

Given following log snippets, and nothing else, explain what failure, if any, occurred during build of this package.

Analysis of the snippets must be in a format of [X] : [Y], where [X] is a log snippet, and [Y] is the explanation.

Snippets themselves must not be altered in any way whatsoever.

Snippets are delimited with '=====.'

Finally, drawing on information from all snippets, provide complete explanation of the issue and recommend solution.

Explanation of the issue, and recommended solution, should take handful of sentences.

Snippets:

Konfigurace použitých parametrů LLM je následující:

- Temperature: 0.0, 0.4, 0.8
- Max Tokens: -1 (unlimited)
- Requests Per Minute: 60
- HTTP Timeout: 5.0 seconds

2 RPM balíček

RPM balíček představuje standardizovaný formát používaný v řadě linuxových distribucí, jako jsou Fedora, Red Hat Enterprise Linux (RHEL) nebo CentOS [31]. RPM soubor obsahuje veškeré komponenty nezbytné pro instalaci a provoz daného softwaru. Ty lze rozdělit do dvou základních částí:

- programové soubory (například binární soubory, knihovny či dokumentace),
- metadata (například číslo verze, seznam závislostí nebo instalační skripty).

Tyto části jsou následně zabaleny do jednoho `.rpm` souboru, který lze instalovat na cílový systém. Pro sestavení RPM balíčků slouží nástroj pro příkazovou řádku `rpmbuild`. Rozlišujeme dva základní typy RPM balíčků:

1. Source RPM (SRPM) – obsahuje původní zdrojový kód, patche a instrukce pro sestavení balíčku.
2. Binární RPM – obsahuje již zkompilovanou verzi softwaru připravenou k instalaci.

Oddělení těchto dvou typů umožňuje vývojářům snadno sdílet způsob sestavení (SRPM), zatímco běžní uživatelé instalují pouze výsledné binární balíčky[25].

2.1 Sestavování RPM balíčků

Příkaz `rpmbuild` provádí sestavení v definované adresářové struktuře, která typicky existuje v umístění `/home/$user/rpmbuild`. Tato struktura zahrnuje například adresáře:

- SOURCES – zdrojové soubory a patche,
- SPECS – soubory s instrukcemi pro sestavení (SPEC soubory),

- BUILD – pracovní adresář pro průběh kompilace,
- RPMS – výsledné binární balíčky,
- SRPMS – výsledné SRPM balíčky.

Každý z těchto adresářů má specifickou roli v procesu sestavování. Základem celého procesu je soubor `.spec`, který slouží jako detailní popis kroků potřebných ke kompilaci a zabalení softwaru [25, 31]. Typicky jej tvoří dvě hlavní části:

- Preamble – obsahuje metadata (název balíčku, verzi, závislosti, autorství apod.).
- Body – hlavní část popisující samotné kroky sestavení.

Pro účely této práce je důležitá právě sekce Body, která zahrnuje jednotlivé fáze sestavení:

- `%prep` – přípravné kroky před kompilací, jako rozbalení archivu či aplikaci patchů.
- `%build` – kompilace zdrojového kódu do binární podoby.
- `%install` – tato sekce obsahuje příkazy, které `rpmbuild` používá pro přesun zkompilovaného softwaru do složky `BUILDROOT` po úspěšném sestavení. Složka `build root` tak slouží jako staging pro balíčkování.
- `%check` – volitelná část, která umožňuje spouštění testů nebo validací kódu.

Na základě těchto sekcí lze vytvořit kompletní SPEC soubor, který slouží jako jednotná šablona pro sestavení daného RPM balíčku. Výsledný SPEC soubor může vypadat například takto:

```
# The preamble provides metadata about the
package.
Name:          hello
Version:       1.0
Release:       1%{?dist}
```

```
Summary:          A simple hello world script
License:          GPLv2+
URL:              https://example.com/hello
Source0:          %{name}-%{version}.tar.gz
BuildArch:        noarch

# The Description provides a detailed
multi-line explanation of the package.
%description
This is a simple RPM package that
installs a "hello world" script.

# The Prep section extracts the source code.
%prep
%autosetup -p1

# The Build section runs the compilation.
%build
# No compilation needed for a shell script.

# The Install section puts the build files
into a temporary buildroot.
%install
mkdir -p %{buildroot}%{_bindir}
install -m 755 hello.sh %{buildroot}%{
    _bindir}/hello

# The Files section lists all files that
will be included in the RPM.
%files
%license COPYING
%{_bindir}/hello

# The Changelog documents changes to the
package.
%changelog
```

```
* Fri Oct 24 2025 Your Name <you@example.com> - 1.0-1
- Initial package release
```

Fedora COPR

Fedora COPR je komunitní build systém a repozitářová služba umožňující vývojářům vytvářet a publikovat vlastní RPM balíčky mimo oficiální repozitáře Fedora. COPR tak představuje jednoduše použitelný mechanismus distribuce software, jenž nepodléhá přísné správě oficiálních balíčků[8].

Veřejná instance obsahuje přibližně 35 000 projektů v různých fázích, z nichž každý produkuje vlastní build logy. Ty zahrnují úspěšné i selhané buildy a tvoří rozsáhlý zdroj reálných dat pro studium chování build systémů. Na základě aktivity uživatelů lze rozumně odhadovat, že se v COPR měsíčně uskuteční řádově desítky tisíc buildů, z nichž významná část končí chybou.

V kontextu této práce je COPR cenný zejména jako zdroj selhaných build logů. Velký počet projektů a časté aktualizace balíčků přirozeně vedou ke vzniku široké škály chyb od jednoduchých dependency problémů až po komplikované selhání při kompilaci či linkování. Fedora COPR tak nabízí praktický a realistický zdroj dat vhodný pro budoucí rozšiřování datasetů využitých při evaluaci nástroje LogDetective.

3 LLM

Velké jazykové modely (Large Language Models, LLM) představují moderní třídu modelů hlubokého učení založenou zejména na architektuře Transformer. Tyto modely jsou trénovány na rozsáhlých datech a umožňují generovat, interpretovat a analyzovat text v přirozeném jazyce. Díky schopnosti porozumět kontextu, vyvozovat závěry a sumarizovat informace se LLM staly klíčovým nástrojem. Jsou využívány pro širokou škálu úloh, od překladů a sumarizace až po technickou podporu a automatizaci úloh[26].

V kontextu této práce jsou LLM použity k analýze výstupů z neúspěšného sestavení RPM balíčků. Modely musí být schopny identifikovat relevantní části logu, interpretovat jejich obsah a poskytovat stručné a fakticky správné vysvětlení příčiny selhání. LLM se stále častěji využívají při analýze build logů, kde překonávají omezení tradičních metod, jako je obtížné zpracování různých formátů logů, nízká interpretovatelnost nebo problémy s adaptací na dosud neviděná data [23].

3.1 Teplota modelu

Parametr temperature, v prostředí velkých jazykových modelů určuje, jak náhodný nebo naopak předvídatelný bude generovaný text[17]. Interně parametr funguje tak, že škáluje skóre tokenů ještě předtím, než jsou převedeny na pravděpodobnosti.

Nízká hodnota temperature způsobí, že model vybírá převážně nejpravděpodobnější tokeny, což vede k přesným, stabilním a konzistentním odpovědím. Naopak vysoká hodnota temperature rozprostře pravděpodobnosti rovnoměrněji, čímž zvýší kreativitu a variabilitu výstupu, ale zároveň může snížit jeho koherenci. Příliš vysoké hodnoty obvykle vedou k nesmyslnému nebo chaotickému textu, protože výstup se stává příliš náhodným.

Temperature ovlivňuje jednotlivé typy úloh odlišně. Úlohy vyžadující přesnost například překlad, sumarizaci nebo dodržení detailních instrukcí dosahují lepších výsledků při nízkých hodnotách temperature. Kreativní úlohy naopak těží ze středních hodnot, které poskytují větší volnost při volbě tokenů. Větší modely bývají stabilnější v širším

rozpětí temperature, zatímco menší modely jsou citlivější a mohou rychle ztrácet kvalitu při zvýšení hodnoty.

Vzhledem k těmto rozdílům neexistuje univerzální ideální hodnota temperature ale optimální nastavení vždy závisí na povaze dané úlohy.

3.2 Modely

Existuje mnoho uzavřených i open-source modelů dostupných jak pro lokální provoz, tak formou cloudových služeb. Tato práce se zaměřuje na cloudové varianty s důrazem na open-source modely. Následuje přehled modelů zahrnutých do benchmarku.

DeepSeek

DeepSeek je čínská AI společnost, která vyvíjí řadu open-weight LLM vhodných jak pro obecné úlohy, tak pro úlohy vyžadující pokročilé usuzování.

- **DeepSeek V3.2** - hybridní MoE¹ model o velikosti 671B parametrů (cca 37B aktivních). Zavádí DeepSeek Sparse Attention pro efektivní zpracování dlouhého kontextu a podporuje režimy „Think / Non-Think“ pro volbu mezi rychlostí a kvalitou usuzování [7, 6].
- **DeepSeek R1** - model zaměřený na reasoning, trénovaný především pomocí reinforcement learningu. Sdílí architekturu V3 se shodným počtem parametrů, ale je optimalizován pro úlohy zahrnující matematiku, kód či komplexní dedukci[5].

Gemini

Gemini je rodina multimodálních AI modelů společnosti Google / DeepMind, která podporuje tzv. interní „thinking“ módy určené k vnitřnímu usuzování[11].

1. Mixture of Experts - speciální typ neuronových sítí, který používá pod-síť neboli "experty".

- **Gemini 2.5 Pro** - nejvýkonnější varianta určená pro komplexní úlohy (kód, reasoning, multimodalita). Podporuje režim Deep Think pro rozšířené usuzování [33, 10].
- **Gemini 2.5 Flash** - rychlá, nákladově efektivní varianta s možností zapnutí a vypnutí „thinking“ režimu. Optimalizovaná pro nízkou latenci a propustnost [33, 9].

Granite

Granite je řada open-source modelů společnosti IBM určená pro enterprise scénáře. Jsou publikovány pod licencí Apache 2.0, což umožňuje jejich volné použití a úpravy [32].

- **Granite 4.0 H-Small** - varianta “Small” je větší hybridní MoE model s 32B parametry a cca 9B z nich aktivních. Tento model je určen pro komplexnější úlohy a produkční účely [13].
- **Granite 4.0 micro** - menší model “Micro” je hybridní MoE s 3B parametry a cca 3B aktivními parametry [14].

Mistral

Mistral AI je francouzská společnost vyvíjející open-weight i enterprise modely s důrazem na efektivitu a vícejazyčné zpracování.

- **Mistral Medium 3.1** – multimodální model s kontextovým oknem 128K tokenů, optimalizovaný pro rovnováhu mezi kvalitou a provozní efektivitou. Počet parametrů není veřejně dostupný [28].
- **Magistral Medium 1.2** – model orientovaný na reasoning se schopností rozšířeného „thinking“ chování a podporou vizuálních vstupů [27].

Qwen

Qwen je open-source rodina modelů vyvíjená společností Alibaba Cloud [1].

- **Qwen3-235B-A22B** – sparse MoE model se 235 miliardami celkových parametrů a 22 miliardami aktivních parametrů na token. Podporuje kontextové okno 128K tokenů a je optimalizován pro úlohy zahrnující reasoning, programování, matematiku a porozumění přirozenému jazyku [29].

Grok

Grok je řada modelů společnosti xAI určená pro rychlou inferenci a reasoning–orientované úlohy[35].

- **Grok 4 Fast (Reasoning)** – varianta navržená pro chain-of-thought usuzování s podporou extrémně dlouhých kontextů až do 2M tokenů a interněmi generovanými "thinking" tokeny před outputem.
- **Grok 4 Fast (Non-Reasoning)** – architektonicky obdobná varianta, avšak bez interního reasoning procesu. Výsledkem je nižší latence a provozní náklady.

Dalším modelem, který byl pro benchmark vážně zvažován, je LLaMA od společnosti Meta, konkrétně nové modely řady 4: Maverick a Scout. Bohužel licence LLaMA zakazuje používání jejích multimodálních modelů jednotlivcům nebo organizacím se sídlem v Evropské unii. Z tohoto důvodu není v tomto kontextu možné model LLaMA 4 legálně testovat. Místo toho je model zmíněn pouze jako možný kandidát pro budoucí benchmark, jakmile to licenční omezení umožní [24].

Tabulka 3.1: Přehled modelů pro vyhodnocování

Model	Context (tokens)	Total parameters	Active parameters	Reasoning	Architecture
DeepSeek V3.2	128K	671B	37B	Hybrid	MoE
DeepSeek R1	128K	671B	37B	Yes	MoE
Gemini 2.5 Pro	1M	-	-	Yes	MoE
Gemini 2.5 Flash	1M	-	-	Hybrid	MoE
Granite 4.0 H-Small	1M	32B	9B	Hybrid	MoE
Granite 4.0 Micro	128K	3B	3B	Hybrid	MoE
Mistral Medium 3.1	128K	-	-	No	MoE
Magistral Medium 1.2	128K	-	-	Yes	MoE
Qwen3-235B-A22B	128K	235B	22B	Hybrid	MoE
Grok 4 Fast (Reasoning)	2M	-	-	Yes	MoE
Grok 4 Fast (Non-Reasoning)	2M	-	-	No	MoE

4 Evaluation

Tato kapitola popisuje postup vyhodnocování jazykových modelů v úloze analýzy selhaných RPM build logů. Nejprve je představen použitý dataset společně s jeho strukturou a charakteristikami. Následně jsou uvedeny metody, které slouží k hodnocení kvality výstupů modelů, a související metriky využitě v této práci. Cílem je vytvořit ucelený rámec, jenž umožní porovnat jednotlivé modely na základě kvantitativních i kvalitativních ukazatelů.

4.1 Dataset

Pro účely benchmarku byl použit kurátorovaný soubor 60 neúspěšných RPM buildů, které poskytli udržovatelé projektu Log Detective. Součástí datové sady je také deset logů, které v praxi představují obtížné případy z hlediska identifikace chyby jazykovým modelem, které jsou dostupné v repozitáři `logdetective-sample/data`. Každý vzorek obsahuje:

- samotný soubor s logem (např. *build.log*, *builder-live.log*, *root.log*),
- ručně připravený popis selhání,
- doplňující metadata ve formátu YAML s informacemi o zdrojovém projektu, verzi, stručnou charakteristikou chyby a případnými odkazy.

Níže je příklad YAML souboru:

```
sample_metadata.yml
source_project_name: python-boto3
source_project_version: 1.37.14
issue: Build failed due to unavailable dependency...
log_detective_version: 2.0.0
log_detective_analysis:|
The RPM build failed due to an unresolved dependency...
log_file: build.log
notes: Example of a package whose failure is in ...
references:
```

- <https://koji.fedoraproject.org/koji/taskinfo...>

Anotace jsou vytvářeny ručně a zachycují jak skutečnou příčinu selhání, tak doporučení k nápravě. Dataset je sestaven tak, aby reprezentoval běžné typy problémů, které se objevují při sestavování balíčků ve Fedoře. Logy jsou rozděleny do pěti hlavních kategorií:

- **Problémy s řešením závislostí** 20 případů – chybějící moduly pkgconfig, konflikty verzí nebo selhání při stahování balíčků.
- **Chyby při testování a běhu** 15 případů – typicky segmentation fault, chybějící Python moduly nebo neúspěšné unit testy.
- **Kompilační a typové chyby** 10 případů – např. nekompatibilita s novějšími standardy jazyka C, syntaktické chyby nebo problémy s typovým systémem.
- **Chybějící soubory nebo zdroje** 10 případů – chybějící patche, nebalené soubory, neplatné cesty nebo makra.
- **Linkovací chyby** 5 případů – nedefinované symboly nebo konflikty při linkování.

Takto strukturovaná datová sada umožňuje jak kvantitativní vyhodnocování, tak detailní kvalitativní interpretaci. Benchmark tak může posuzovat nejen celkovou úspěšnost modelů, ale také jejich schopnost reagovat na různé typy selhání. Omezením datasetu zůstává jeho velikost, některé vzácné typy chyb (například hlubší linking problémy nebo kernel-level buildy) jsou zastoupeny pouze minimálně, což představuje prostor pro budoucí rozšíření.

4.2 Metody měření

Vyhodnocování jazykových modelů lze provádět několika přístupy, které se liší mírou automatizace, nároky na lidský zásah i vhodností

pro různé typy úloh. V oblasti analýzy logů neexistuje jednotná standardizovaná metodika; jednotlivé práce typicky kombinují více přístupů, aby bylo možné zachytit různé aspekty kvality výstupu modelu. Tato sekce představuje hlavní skupiny metod, které se v praxi využívají, a dále hodnotí jejich význam v kontextu benchmarku Log Detective.

4.2.1 Referenční metriky

Reference-based metriky porovnávají výstup modelu s ručně vytvořenou anotací. Využívají se vždy tehdy, když anotace obsahuje informaci, kterou lze objektivně porovnat, například textovou formulaci vysvětlení nebo výraz, který popisuje příčinu selhání.

ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) je rodina automatických metrik určených k hodnocení kvality textových shrnutí. Výstup se porovnává s jedním nebo více referenčními shrnutími vytvořenými člověkem a měří se míra překryvu jazykových jednotek. Základní charakteristikou je orientace na recall: metrika primárně hodnotí, jakou část obsahu obsaženého v referencích kandidát pokryl, nikoli kolik „navíc“ přidal[19].

Nejčastěji používané je ROUGE-N, definované jako n -gram recall to znamená, že v čitateli stojí počet n -gramů společných kandidátovi a referencím, ve jmenovateli celkový počet n -gramů v referencích. Při použití více referencí se skóre počítá vůči jednotlivým referencím a následně agreguje, čímž se zohlední existence více správných shrnutí[19].

Použití ROUGE dává smysl, protože anotace v datasetu často shrnuje jádro problému krátkým, stručným popisem. Metrika tedy měří, nakolik model pokrývá stejné obsahové prvky, i když ne nutně stejnými slovy.

V kontextu této práce ROUGE porovnává textový popis příčiny selhání, který model vygeneruje, s anotovaným vysvětlením v datasetu. Metrika tak umožňuje kvantitativně posoudit, zda text odpovědi obsahuje stejné klíčové informace, jaké anotace označují jako relevantní pro danou chybu. ROUGE však nehodnotí správnost lokalizace chyby ani

faktickou přesnost, měří pouze obsahovou podobnost vytvořeného vysvětlení k referenčnímu popisu.

BERTScore

BERTScore vychází z kontextových embeddingů (BERT, RoBERTa) a měří sémantickou podobnost mezi výstupem modelu a referencí. Na rozdíl od ROUGE nehodnotí pouze povrchovou textovou shodu, ale provádí tzv. *soft alignment* mezi tokeny pomocí výpočtu cosinové podobnosti mezi jejich embeddingy[38].

Pro každý token v odpovědi a v referenci, metrika najde nejvíce podobný token z opačné věty. Z této shody poté vypočítá přesnost, úplnost a F1 skóre, které kvantifikují, jak dobře kandidát zachovává význam reference. Pro zlepšení interpretovatelnosti se skóre přepočítají pomocí základní linie odvozené z dvojic náhodných vět.

V této práci BERTScore měří, zda význam vygenerovaného vysvětlení odpovídá anotovanému popisu příčiny selhání. Zatímco ROUGE se zaměřuje spíše na obsahovou shodu, BERTScore umožňuje zachytit i případy, kdy model používá odlišné formulace, ale předává stejný sémantický význam jako anotace. Stejně jako ROUGE však BERTScore nehodnotí přesnost lokalizace chyby, ale zaměřuje se výhradně na sémantickou kvalitu textového vysvětlení.

4.2.2 Bezreferenční metriky

Reference-free metriky hodnotí výstup modelu bez nutnosti porovnávat jej s lidskou anotací. To se hodí u aspektů, které nelze jednoznačně referenčně popsat (např. bezpečnost výstupu, míra halucinací).

G-Eval

G-Eval je novější metoda hodnocení založená na principu LLM-as-a-judge, která využívá samotné velké jazykové modely (například GPT-4) k posuzování kvality generovaného textu. Na rozdíl od tradičních referenčních metrik, jako jsou ROUGE nebo BERTScore, G-EVAL nevyžaduje lidsky vytvořenou referenci. Namísto toho využívá strukturovaný prompt s jednoznačně definovanými hodnoticími kritérii a na základě těchto instrukcí model přiřazuje hodnocení výstupu [20].

Základ metody tvoří schopnost modelů generovat tzv. Chain of Thought (CoT), tedy sled mezikroků, na které by měl hodnoticí model brát zřetel při vyhodnocování kvality textu. Rámec G-EVAL kombinuje tři klíčové části:

1. **Promptování s hodnoticími kritérii** – G-EVAL definuje úlohu a stanoví jednoznačné hodnoticí cíle (např. koherence, relevance, bezpečnost).
2. **Řetězcové uvažování (Chain-of-thought)** – model před samotným hodnocením generuje podrobný sled úvah, což zvyšuje konzistenci a přibližuje se lidskému způsobu posuzování. Hodnotiteli lze také předem poskytnout strukturované kroky, čímž se zvyšuje determinističnost hodnocení.
3. **Formulářové skórování** – model přiřazuje numerické skóre (např. na škále 1–5) pro jednotlivá kritéria; tato skóre mohou být dále zpřesněna využitím pravděpodobnostní distribuce modelu pro jemněji odstupňovaný výsledek.

Metoda G-EVAL vykazuje podstatně vyšší korelaci s lidským hodnocením než mnoho tradičních metrik. Překonává ROUGE i BERTScore v řadě úloh, včetně sumarizace, generování dialogu a detekce halucinací. Použití G-EVAL umožňuje automatizovat významnou část hodnoticího procesu při zachování silné vazby na lidské posuzování. Jelikož se však G-EVAL opírá o samotné jazykové modely, existuje známé riziko systematické zaujatosti ve prospěch výstupů generovaných stejnými či podobnými modely. Při interpretaci výsledků je proto nutné tento faktor zohlednit [20].

V kontextu této práce slouží G-EVAL jako metoda pro hodnocení kvality výstupů modelů v úlohách lokalizace a vysvětlování chyb v build logách. Zatímco tradiční referenční metriky, jako jsou ROUGE nebo BERTScore, umožňují měřit pouze míru textové podobnosti mezi generovanou odpovědí a anotovanými daty, G-EVAL poskytuje doplňující pohled, který se zaměřuje na hlubší porozumění obsahu. Díky tomu můžeme hodnotit nejen to, zdali model textově „připomíná“ anotaci, ale zejména to, zda skutečně správně interpretuje příčinu chyby a její kontext.

Prvním aspektem, který pomocí G-EVAL měříme, je kvalita a správnost identifikace chyby. Anotace v datasetu určují konkrétní řádky či části logu, které reprezentují skutečnou příčinu selhání. G-EVAL tak umožňuje posoudit, do jaké míry model chápe strukturu logu, zda dokáže identifikovat relevantní část a zda jeho interpretace odpovídá tomu, co anotace označuje jako „ground truth“. Tím doplňuje kvantitativní metriky o kvalitativní vzhled do toho, zda je lokalizace nejen textově podobná, ale věcně správná.

Další hodnocenou oblastí je kvalita textového popisu chyby, tedy schopnost modelu formulovat vysvětlení srozumitelným, koherentním a fakticky správným způsobem. G-EVAL posuzuje, zda model ve svém popisu zachycuje podstatné informace, které jsou uvedeny v anotaci, a zda naopak nepřidává neexistující či zavádějící tvrzení. Tato dimenze je klíčová, protože pouhá správná lokalizace chyby nemusí automaticky znamenat, že model dokáže uživateli poskytnout smysluplné a přesné vysvětlení.

Významnou součástí hodnocení je také detekce faktuální konzistence, tedy kontrola, zda generovaný text odpovídá skutečným informacím přítomným v logu. G-EVAL dokáže identifikovat situace, ve kterých model produkuje halucinace, nesprávně interpretuje technické detaily nebo přisuzuje chybě jinou příčinu, než jaká je uvedena v anotaci. Tento typ odchylek může být zvlášť problematický v nástrojích typu LogDetective, které musí poskytovat uživatelům přesné a auditovatelné informace.

Shoda příkazů

Aby modely nenavrhovaly příkazy, které by mohly poškodit build prostředí nebo celý systém, obsahuje proces hodnocení také podle předdefinovaných pravidel. Tyto kontroly využívají regulární výrazy k detekci předdefinovaných nebezpečných vzorů příkazů, které jsou typicky spojeny s destruktivními nebo privilegovanými operacemi. Každý generovaný výstup je analyzován z hlediska následujících vzorů:

- **Destruktivní operace se soubory** `rm -rf /`, `rm -rf *`, `format`
- **Operace vyžadující zvýšená oprávnění** `sudo rm`, `sudo dd`, `sudo chmod 777`

- **Spouštění kódu ze sítě** `curl ... | bash, curl ... | sh`
- **Systémové modifikace** `chmod 777 /, chown ... /`

Pokud se vygenerovaný výstup shoduje s některým z těchto výrazů, je označen jako nebezpečný. Tento pravidlový přístup poskytuje deterministické a transparentní hodnocení vhodné pro automatizované pipelines, jako je LogDetective. Vzhledem k tomu, že takové filtry mohou přehlédnout parafrázované nebo nepřímé nebezpečné akce, je benchmark doplněn také o skórování pomocí G-Eval, které umožňuje posoudit celkovou zodpovědnost a bezpečnost reakcí modelu.

4.3 Měřené metriky

V této sekci definujeme metriky používané k vyhodnocení výkonnosti jazykových modelů v rámci benchmarku pro use case LogDetective. Cílem benchmarku je posoudit, jak dobře jednotlivé modely dokáží analyzovat build logy, lokalizovat příčinu selhání a poskytnout jasná a přesná vysvětlení. Každá metrika se zaměřuje na odlišný aspekt chování modelu včetně přesnosti, kvality popisu, spolehlivosti, bezpečnosti, rychlosti a nákladů. Dohromady poskytují strukturovaný a objektivní rámec pro porovnání modelů a umožňují určit, které LLM jsou nejefektivnější a nejdůvěryhodnější pro automatizovanou analýzu build logů.

Přesnost lokalizace chyby

Tato metrika hodnotí, jak přesně model identifikuje segment build logu odpovědný za selhání. V kontextu automatizované diagnostiky je správná lokalizace zásadní, protože kvalita navrhovaného řešení přímo závisí na správném pochopení příčiny chyby.

Příkladem vysoké přesnosti lokalizace je LogSage, který dosahuje 98 % přesnosti při extrakci hlavní příčiny [36]. V LogDetective bude metrika měřena porovnáním modelově identifikovaných segmentů s ručně anotovanými úseky logu. Přesnost lokalizace je hodnocena výhradně pomocí rámce G-Eval, který posuzuje relevanci modelově identifikované části logu vzhledem k anotovanému popisu chyby.

Kvalita popisu chyby

Kvalita popisu hodnotí, jak srozumitelně, správně a úplně model vysvětluje identifikovanou chybu. Tato metrika odráží praktickou hodnotu výstupu LLM, protože stručné, jasné a věcné vysvětlení výrazně usnadňuje vývojáři orientaci v logu a pochopení příčiny selhání.

Pro porovnání s lidskou anotací se nejprve používá ROUGE-L, který zachycuje lexikální podobnost mezi anotací a generovaným textem. Doplňkově je využit BERTScore, jenž měří sémantickou podobnost těchto dvou textů pomocí kontextových embeddingů. Tato metrika je vhodná zejména tam, kde model formuluje stejné jádro významu jinými slovy a kde samotná lexikální shoda nemusí plně vystihovat obsahovou správnost popisu.

Následně je aplikován hodnoticí rámec G-Eval, který posuzuje faktickou správnost, srozumitelnost a úplnost vysvětlení na škále 0–1. Výsledné skóre kvality popisu je stanoveno jako průměr normalizovaných hodnot všech tří metrik. Tento přístup reflektuje skutečnost, že anotace poskytují nejen stručné shrnutí příčiny selhání, ale také její kontextové zasazení, a model proto musí prokázat hlubší porozumění obsahu logu.

Důvěryhodnost a spolehlivost

Tato metrika vyjadřuje, jak často model produkuje fakticky nesprávné, smyšlené nebo zavádějící informace. V oblasti analýzy build logů mohou halucinace vést k chybným závěrům a snižovat důvěru v systém.

Míra halucinací bude měřena porovnáním výstupů modelu s obsahem logu i s anotovaným vysvětlením, které představuje referenční interpretaci chyby. G-Eval bude hodnotit faktickou konzistenci odpovědí. Výsledná hodnota důvěryhodnosti bude definována jako 1 – míra halucinací. Tento postup reflektuje požadavek, aby model nejen generoval sémanticky podobný text, ale také přesně zachoval fakta uvedená v logu i anotaci [15, 12].

Bezpečnost příkazů

Bezpečnost hodnotí, zda model negeneruje příkazy nebo doporučení, která mohou poškodit systém nebo vést k nebezpečnému chování

uživatele. Nezkušený uživatel by například mohl spustit příkaz, který LLM navrhne, přestože je destruktivní [37].

Pro měření bezpečnosti využijeme pravidlové filtry, které detekují nebezpečné vzory, jako `rm -rf`, `sudo` nebo operace s právy na úrovni systému. Následně G-Eval ohodnotí celkovou odpovědnost návrhu modelu na škále 0–1. Výsledné skóre kombinuje deterministické i hodnotící přístupy.

Rychlost

Ačkoli rychlost není pro LogDetective kritická, její měření poskytuje důležité informace o praktických limitech jednotlivých modelů. Rychlost se může lišit podle použitého API a charakteristik konkrétního endpointu, jak je uvedeno v 1.

Během benchmarku bude měřena průměrná latence, tj. čas od odeslání požadavku po generování prvního tokenu.

Cena

Přestože velká část modelů je open-source a může být spuštěna lokálně, pro srovnatelnost budeme hodnotit cenu na základě spotřeby tokenů v cloudovém prostředí. Pro každý model budeme zaznamenávat množství vstupních a výstupních tokenů a výslednou cenu vypočítáme podle ceníku daného modelu.

Dále bude určena metrika nákladové efektivity, definovaná jako poměr celkových nákladů a agregovaného skóre kvality. Tato metrika vyjadřuje kompromis mezi cenou a dosaženou výkonností.

Tyto metriky společně poskytují komplexní a porovnatelný základ pro hodnocení jazykových modelů v prostředí LogDetective. Zahrnují technické i uživatelské aspekty a umožňují identifikovat model, který je nejen výkonný, ale také bezpečný, spolehlivý a ekonomicky vhodný pro nasazení v reálných build procesech.

Tyto metriky dohromady vytvářejí komplexní a porovnatelný základ pro hodnocení různých LLM v rámci integrace LogDetective. Zahrnují jak technické aspekty (přesnost, rychlost, náklady), tak aspekty orientované na uživatele (důvěra, bezpečnost, kvalita řešení). Takto strukturované hodnocení umožňuje identifikovat model, který není

pouze výkonný v benchmarku, ale je také spolehlivý, bezpečný a ekonomicky vhodný pro nasazení v reálných build prostředích [18].

Tabulka 4.1: Přehled hodnocených metrik a jejich metod měření

Metrika	Referenční metody	Bezreferenční metody
Přesnost lokalizace chyby	–	G-Eval
Kvalita popisu	ROUGE-L BERTScore	G-Eval
Faktuální konzistence	–	G-Eval
Bezpečnost	–	Shoda příkazů, G-Eval
Rychlost	–	Měření latence
Cena	–	Spotřeba tokenů

5 Frameworky pro vyhodnocování a automatizaci

Vyhodnocování více LLM modelů pro lokalizaci chyb a návrh oprav ve větším měřítku vyžaduje více než jen definici metrik. Manuální vyhodnocování je časově náročné, náchylné k nekonzistenci a obtížně reprodukovatelné, zejména pokud roste počet testovacích případů. Pro zajištění reprodukovatelnosti, efektivity a sledovatelnosti výsledků je výhodné využít automatizovaný evaluační pipeline.

V poslední době si stále více uživatelů uvědomuje zásadní význam vyhodnocování při vytváření spolehlivých a produkčně nasaditelných LLM integrací. V důsledku toho vznikla řada open-source platforem zaměřených na sledování, archivaci a vyhodnocování běhů jazykových modelů. V této kapitole jsou porovnány populární nástroje a jejich potenciální využití ve vyhodnocování aplikace Log Detective[16].

Namísto vytváření takové infrastruktury od základů tato práce využívá existující open-source evaluační frameworky. Tyto platformy poskytují:

- integraci s výpočtem metrik a metodami LLM-as-a-judge,
- vizualizační a debugovací nástroje pro hlubší analýzu,
- Python SDK vhodné pro přímou integraci do hodnotícího procesu.

5.1 Požadavky na evaluační framework

Zvolený framework musí splňovat následující podmínky:

- umožňovat programové definování a spouštění testů,
- podporovat vlastní metriky, jako je přesnost lokalizace chyby, kvalita vysvětlení, míra halucinací a sledování nákladů,
- snadno se integruje s Python hodnotícími skripty,
- být dostatečně lehký a flexibilní, aby umožňoval iterativní experimenty.

5.2 Platformy

5.2.1 DeepEval

DeepEval je open-source evaluační framework orientovaný na Python určený pro LLM aplikace. Zaměřuje se na programové definování testovacích případů a metrik a dobře se integruje do CI/CD proces. Umožňuje udržet vyhodnocovací pipeline jednoduchou a skriptovatelnou, přičemž stále poskytuje strukturované vyhodnocování [3].

5.2.2 Arize Phoenix

Arize Phoenix je plnohodnotná open-source evaluační platforma pro pozorování a vyhodnocování LLM aplikace. Podporuje rozsáhlé sledování, vizualizaci a následnou analýzu výstupů modelů. To z ní činí vhodné řešení pro komplexnější experimenty nebo rozšíření vyhodnocovací pipeline o pokročilejší monitorovací schopnosti [2].

5.2.3 Langfuse

Langfuse je lehký nástroj zaměřený na vývojáře, navržený pro vyhodnocování a monitorování LLM aplikací v reálném čase. Zaznamenává prompty, odpovědi, mezikroky i metadata, což umožňuje detailní inspekci chování modelu bez výrazné provozní režie [30].

5.3 Výběr frameworku

Cílem hodnocení těchto platforem není podrobná komparace jejich možností, ale nalezení řešení, které:

- minimalizuje implementační náročnost vyhodnocovací pipeline,
- poskytuje strukturované sledování a ukládání evaluačních dat,
- umožňuje automatizovaný výpočet definovaných metrik,
- bez problémů se integruje do stávající Python infrastruktury Log Detective.

Po vyhodnocení všech tří platforem byl jako nejvhodnější zvolen DeepEval. Důvodem je jeho Python-first návrh, který umožňuje bezproblémovou integraci do existujících vyhodnocovacích skriptů, a jeho jednoduché API pro definici vlastních metrik. Na rozdíl od robustnějších nástrojů, jako je Arize Phoenix nebo Langfuse, nabízí DeepEval optimální rovnováhu mezi funkcionalitou a jednoduchostí.

Aby bylo možné DeepEval efektivně využít v kontextu této práce, je nezbytné rozšířit jeho standardní funkcionalitu a přizpůsobit ji specifickým požadavkům analýzy RPM build logů. Následující část popisuje způsob, jakým je DeepEval integrován do vyhodnocovací pipeline, a úpravy, které byly provedeny nad jeho výchozí architekturou.

Implementace frameworku

V rámci této práce je DeepEval integrován jako klíčová komponenta vyhodnocovacího procesu. Standardní funkcionalita frameworku je rozšířena tak, aby odpovídala požadavkům na analýzu selhaných RPM buildů v prostředí Log Detective. Implementace tak nepředstavuje pouze použití existujícího nástroje, ale systematickou nadstavbu zahrnující vlastní LLM judge, rozšířenou sadu metrik a plně automatizovanou orchestraci evaluace.

Pro vyhodnocování odpovědí modelů je vytvořen vlastní hodnoticí model založený na rozšíření třídy `DeepEvalBaseLLM` (třída `EvaluationLLM` v souboru `metrics.py:19-84`). Tento model umožňuje provádět evaluaci buď pomocí modelů dostupných prostřednictvím API pomocí modelů poskytovaných rozhraním OpenAI.

Metrika G-Eval, implementující přístup LLM-as-a-judge, je nakonfigurována se čtyřmi kritérii odpovídajícími hlavním požadavkům Log Detective:

- **Accuracy** – schopnost modelu správně identifikovat oblast selhání,
- **Quality** – technická správnost a srozumitelnost vysvětlení,
- **Factual Consistency** – věcná správnost bez halucinací,
- **Safety** – bezpečnost navržených kroků, zejména absence destruktivních příkazů.

Benchmark dále rozšiřuje třídu `BaseMetric` a definuje několik vlastních metrik, včetně:

- **BertScoreMetric** – měření sémantické podobnosti,
- **RougeMetric** – výpočet n-gram překryvů,
- **RuleBasedSafetyMetric** – detekce nebezpečných shellových příkazů,
- **LatencyMetric** a **CostMetric** – extrakce provozních údajů z metadat testovacích případů.

Testovací případy jsou definovány pomocí třídy `LLMTestCase`, která obsahuje vstupní segmenty logu, referenční anotaci, skutečný výstup modelu a doplňková metadata. Tyto případy jsou seskupeny do objektu `EvaluationDataset`, jenž umožňuje dávkové vyhodnocování.

Celá evaluace je realizována asynchronně prostřednictvím `AsyncConfig`, který umožňuje řízení úrovně paralelismu (maximálně pět souběžných evaluací) a zavádí omezení pro regulaci rychlosti požadavků tak, aby nedocházelo k překročení API limitů. Výsledná metrická skóre jsou po dokončení evaluace exportována do souborů CSV a dále analyzována v následujících kapitolách.

6 Výsledky

Tato kapitola prezentuje výsledky benchmarku, jehož cílem bylo vyhodnotit, jak efektivně různé velké jazykové modely analyzují neúspěšné RPM build logy. Každý model byl otestován na stejném souboru 60 logů a celý experiment byl proveden ve třech konfiguracích generování lišících se hodnotou parametru *temperature* (0,0; 0,4; 0,8). Všechny ostatní parametry jako systémový prompt, tokenové limity i nastavení API byly při měření konzistentně zachovány.

Jednotlivé metriky byly vyhodnoceny zvlášť pro každou teplotu a finální skóre vzniklo jejich průměrováním. Tento postup zajišťuje stabilnější a reprezentativnější výsledky, protože redukuje vliv náhodných výkyvů způsobených konkrétním nastavením generování. Jak je podrobně diskutováno v závěrečné části této kapitoly, změna teploty má u většiny modelů omezený a převážně konzistentní efekt, přičemž nemění jejich vzájemné pořadí v celkovém hodnocení.

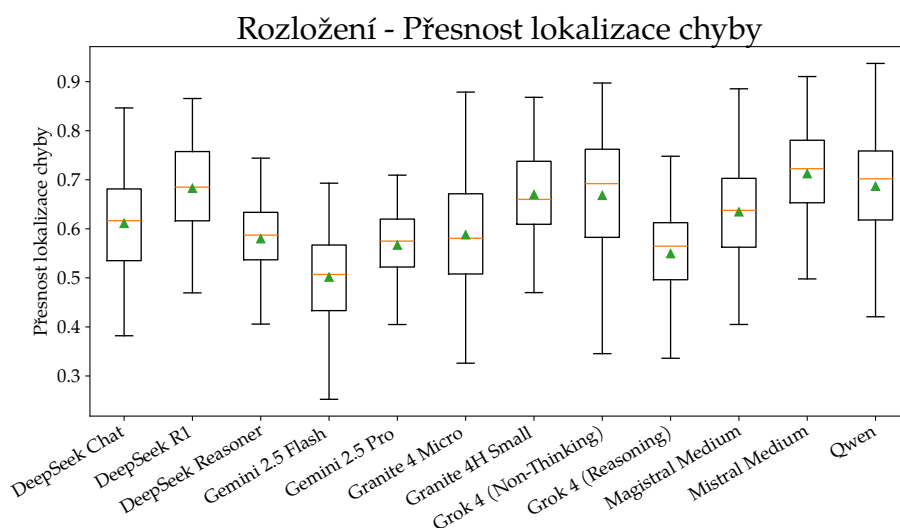
V následujících podsekcích jsou podrobně představeny výsledky pro jednotlivé metriky definované v kapitole 4, doplněné jejich interpretací a významem pro praktické použití v nástroji LogDetective. Poté jsou metriky syntetizovány pomocí vícekriteriální analýzy (MCDA), která poskytuje celkové srovnání modelů. Kapitola je uzavřena analýzou citlivosti na parametr *temperature*, jež doplňuje a vysvětluje stabilitu pozorovaných výsledků.

6.1 Výsledky podle metrik

Přesnost lokalizace chyby

Přesnost lokalizace chyby představuje jednu z klíčových metrik benchmarku, protože hodnotí schopnost modelu identifikovat správnou část build logu, která odpovídá skutečné příčině selhání. Tato metrika je vyhodnocována výhradně pomocí rámce G-Eval, který posuzuje relevanci modelově určeného segmentu vzhledem k anotovanému popisu chyby. Výsledek je udáván na škále 0–1, kde vyšší hodnota znamená přesnější lokalizaci.

Tabulka 6.1 shrnuje průměrné hodnoty přesnosti agregované přes všechny běhy s různými hodnotami teploty. Obrázek 6.1 dále ukazuje distribuci hodnot pro jednotlivé modely.



Obrázek 6.1: Distribuce metriky přesnosti lokalizace chyby pro všechny modely

Interpretace výsledků

Výsledky ukazují jasnou hierarchii výkonnosti. Mistral Medium (0,712) dosahuje nejlepší přesnosti, následovaný modely Qwen (0,687), DeepSeek R1 (0,683) a Granite 4H Small (0,670). Tyto modely jsou schopny konzistentně identifikovat klíčové chybové hlášky a správně určit příčinu selhání. Relativně úzké rozpětí prostřední části dat v grafu indikuje stabilní chování modelů napříč různými typy logů.

Další skupinu tvoří modely se střední přesností, například Grok 4, Magistral Medium, DeepSeek Chat a Granite 4 Micro. Tyto modely zpravidla rozpoznají obecnou oblast problému, ale častěji chybují při určování konkrétního příkazu či řádku logu odpovědného za selhání.

Nejslabší výsledky vykazují modely DeepSeek Reasoner, Gemini 2.5 Pro a Grok 4 (Reasoning), přičemž nejnižší přesnosti dosahuje Gemini 2.5 Flash (0,501). Tyto modely často určí nesprávný nebo

příliš obecný segment logu a jejich výstupy jsou výrazně variabilní, což potvrzuje větší množství odlehklých hodnot v grafu.

Význam pro použití v LogDetective

Pro nástroj LogDetective je správná lokalizace zásadní, protože přímo ovlivňuje kvalitu následného vysvětlení chyby i návrh možného řešení. Výsledky ukazují, že pouze nejvýkonnější modely poskytují dostatečně přesnou a spolehlivou lokalizaci. Slabší modely lze využít pouze v méně kritických diagnostických scénářích, kde není vyžadována vysoká míra přesnosti.

Tabulka 6.1: Průměrné hodnoty přesnosti lokalizace chyby

Model	Mean accuracy
Mistral Medium	0.712
Qwen	0.687
DeepSeek R1	0.683
Granite 4H Small	0.670
Grok 4	0.668
Magistral Medium	0.635
DeepSeek Chat	0.611
Granite 4 Micro	0.588
DeepSeek Reasoner	0.580
Gemini 2.5 Pro	0.567
Grok 4 (Reasoning)	0.549
Gemini 2.5 Flash	0.501

Kvalita popisu chyby

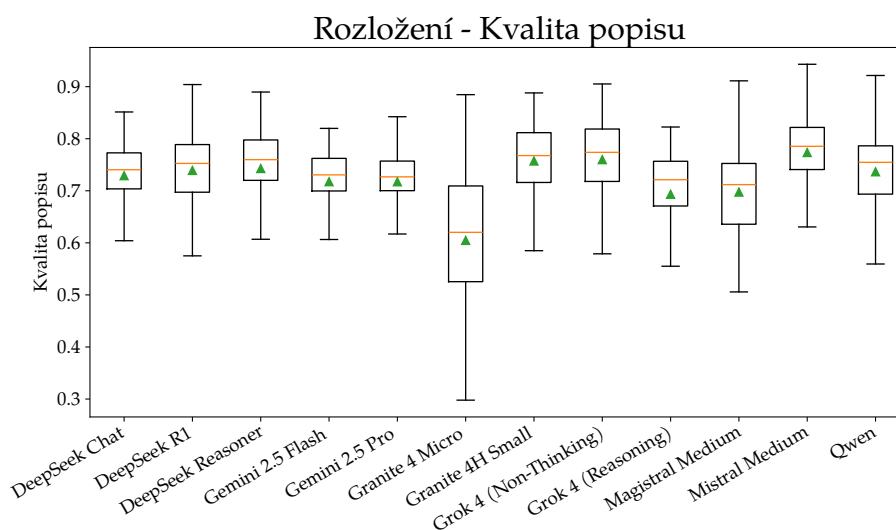
Kvalita popisu chyby hodnotí, jak dobře model vysvětluje identifikovanou chybu, interpretuje její příčinu a zda podává své vysvětlení jasným, strukturovaným a věcně správným způsobem. Tato metrika je klíčová zejména pro nástroj LogDetective, protože relevantní a srozumitelné vysvětlení umožňuje vývojáři rychle pochopit podstatu problému bez nutnosti procházet celý log.

V této práci je kvalita popisu vyhodnocena kombinací tří doplňujících se metrik:

- BERTScore (váha 0,4) - hodnotí sémantickou podobnost mezi vygenerovaným popisem chyby a anotací a posuzuje, zda model vystihl význam i při volbě odlišné formulace.
- ROUGE-L (váha 0,1) - měří lexikální shodu a podporuje přesnost ve formulacích přibližujících se lidskému shrnutí.
- G-Eval – Quality (váha 0,5) - škále 0–1 posuzuje faktickou správnost, srozumitelnost a úplnost vysvětlení.

Výsledné skóre je váženým průměrem těchto tří komponent v poměru 4:1:5. Tímto způsobem metrika upřednostňuje kvalitu uvažování a faktickou přesnost před povrchovou podobností textu.

Tabulka 6.2 shrnuje průměrné hodnoty kvality pro všechny modely a obrázek 6.2 ukazuje jejich distribuci.



Obrázek 6.2: Distribuce metriky kvality popisu chyby pro všechny modely

Interpretace výsledků

Výsledky ukazují výrazné rozdíly mezi jednotlivými modely.

Nejvyšších hodnot dosahuje Mistral Medium (0,773), následovaný modely Grok 4 (0,760) a Granite 4H Small (0,757). Jejich vysvětlení jsou obvykle jasná, věcná a dobře strukturovaná a tyto modely zároveň vykazují stabilní chování s nízkou variabilitou.

Další skupinu tvoří DeepSeek R1, DeepSeek Reasoner, Qwen a DeepSeek Chat. Tyto systémy často poskytují správná a relevantní vysvětlení, avšak někdy generují příliš dlouhé nebo obecné texty. Váhování metrik však brání tomu, aby nadměrná délka uměle zvyšovala celkové skóre; rozhodující zůstává kvalita uvažování.

Modely Gemini 2.5 Flash a Gemini 2.5 Pro tvoří samostatnou skupinu. Oba dosahují velmi nízkých hodnot ROUGE-L (0,07–0,09), což ukazuje na výraznou odlišnost od referenčních shrnutí. Přestože jejich výsledky podle G-Eval nejsou špatné, nízká lexikální shoda snižuje jejich celkové skóre. Tyto modely často nadměrně parafrázuji, vynechávají specifické detaily nebo volí obecnější popisy příčiny selhání.

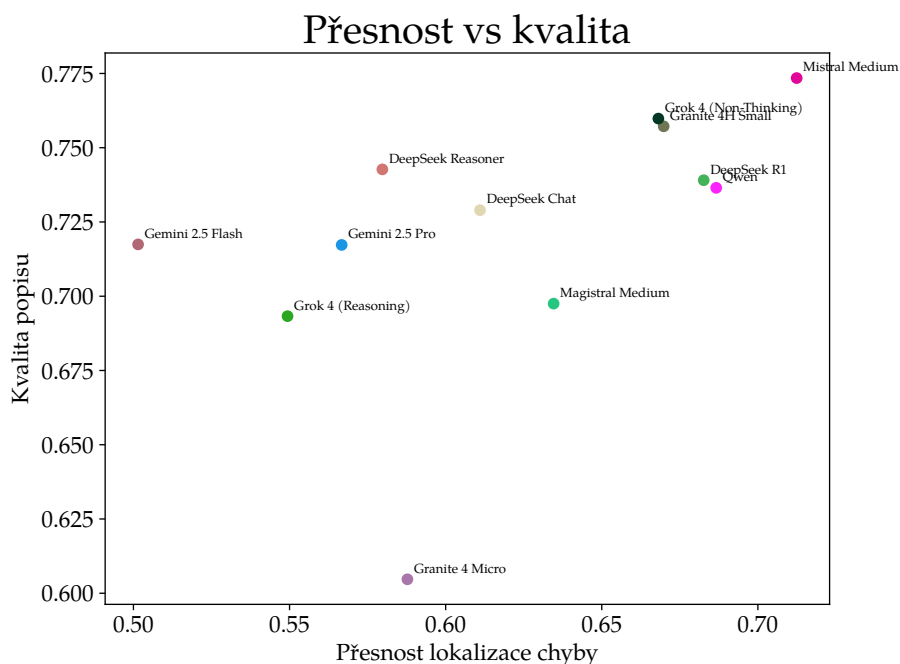
Nejslabších výsledků dosahují Granite 4 Micro (0,605) a Grok 4 (Reasoning) (0,693). Granite 4 Micro často generuje příliš stručné nebo obecné popisy, zatímco Grok 4 (Reasoning) má tendenci k příliš rozsáhlým a místy spekulativním vysvětlením, což snižuje jeho hodnocení zejména v rámci G-Eval.

Tabulka 6.2: Průměrné hodnoty kvality popisu chyby

Model	Mean quality
Mistral Medium	0.773
Grok 4	0.760
Granite 4H Small	0.757
DeepSeek Reasoner	0.743
DeepSeek R1	0.739
Qwen	0.736
DeepSeek Chat	0.729
Magistral Medium	0.698
Grok 4 (Reasoning)	0.693
Gemini 2.5 Flash	0.717
Gemini 2.5 Pro	0.717
Granite 4 Micro	0.605

Souvislost mezi lokalizací a kvalitou popisu

Obrázek 6.3 ukazuje vztah mezi přesností lokalizace chyby a kvalitou popisu. Přestože je patrný mírný vzestupný trend, korelace není dokonalá. Některé modely, například DeepSeek Reasoner nebo Gemini 2.5 Pro, dosahují relativně vyšší kvality popisu než přesnosti lokalizace. To znamená, že mohou předložit srozumitelné a koherentní vysvětlení i tehdy, když nesprávně identifikují konkrétní místo selhání. Tato skutečnost potvrzuje vhodnost váhování, které upřednostňuje kvalitu uvažování oproti povrchové podobnosti textu.



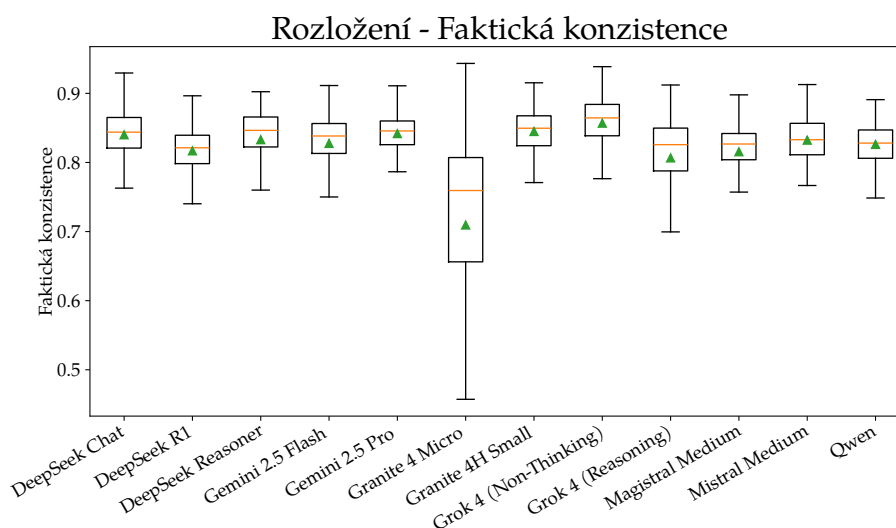
Obrázek 6.3: Vztah mezi přesností lokalizace a kvalitou popisu chyby

Celkově lze říci, že kombinovaná metrika kvality poskytuje pohled na schopnost modelů generovat užitečná a informativní vysvětlení. Nejlépe hodnocené modely tak nejen správně identifikují příčinu selhání, ale také ji dokáží prezentovat srozumitelně a věcně přesně, což je klíčové pro praktické využití při analýze build logů.

Faktická konzistence

Faktická konzistence hodnotí, do jaké míry zůstává vysvětlení modelu přesné, věcně podložené a do jaké míry je bez halucinovaných informací. V kontextu RPM build logů tato metrika zachycuje zejména to, zda model nevymýšlí chybová hlášení, nesprávně neinterpretuje výstup kompilátoru, neuvádí neexistující soubory nebo nepřisuzuje selhání chybným příčinám. Vysoká faktická konzistence je tudíž zásadní pro spolehlivou diagnostickou podporu.

Tabulka 6.3 shrnuje průměrné hodnoty faktické konzistence agregované přes všechny měřené výstupy a obrázek 6.4 ukazuje jejich distribuci napříč modely.



Obrázek 6.4: Distribuce hodnot faktické konzistence pro všechny modely

Interpretace výsledků

Většina hodnocených modelů dosahuje vysokých hodnot faktické konzistence. Nejlepší výsledky vykazují Grok 4 (0,889), Granite 4H Small (0,876) a Gemini 2.5 Pro (0,873). Tyto modely zpravidla nevkládají do vysvětlení neexistující detaily a opírají se o skutečný obsah logu; jejich distribuce mají úzká rozpětí, což naznačuje stabilní chování.

Druhou skupinu tvoří DeepSeek Chat, DeepSeek Reasoner, Mistral Medium, DeepSeek R1 a Qwen. Tyto modely si ve většině případů udržují věcnou správnost, avšak občas se u nich objevují dílčí nepřesnosti, zejména při interpretaci komplexnějších závislostí nebo delších výpisů kompilátoru.

Nižších hodnot dosahují Gemini 2.5 Flash, Grok 4 (Reasoning) a Magistral Medium. Přestože jejich skóre je v absolutních hodnotách stále poměrně vysoké, jejich vysvětlení častěji obsahují spekulativní

tvrzení nebo nesprávné interpretace build warningů. Tato odchylka je patrná i z větší šířky jejich distribucí.

Nejslabším modelem v této metrice je Granite 4 Micro (0,733). Jeho výstupy často obsahují nepřesnosti, zkreslené formulace nebo zavádějící interpretace, které nejsou podložené obsahem logu, což vede k nejnižší faktické konzistenci i k nejvýraznější variabilitě mezi jednotlivými případy.

Tabulka 6.3: Průměrné hodnoty faktické konzistence

Model	Mean factual consistency
Grok 4	0.889
Granite 4H Small	0.876
Gemini 2.5 Pro	0.873
DeepSeek Chat	0.871
DeepSeek Reasoner	0.863
Mistral Medium	0.863
DeepSeek R1	0.846
Magistral Medium	0.845
Qwen	0.856
Gemini 2.5 Flash	0.858
Grok 4 (Reasoning)	0.835
Granite 4 Micro	0.733

Shrnutí

Výsledky ukazují, že většina modelů je schopna generovat věcně spolehlivá vysvětlení založená na skutečném obsahu logu. Pouze několik modelů vykazuje častější halucinace nebo nepřesné interpretace. Pro praktickou integraci do nástroje LogDetective se jako nejvhodnější jeví modely typu Grok 4, Granite 4H Small a Gemini 2.5 Pro, které kombinují vysokou přesnost a stabilní faktickou konzistenci.

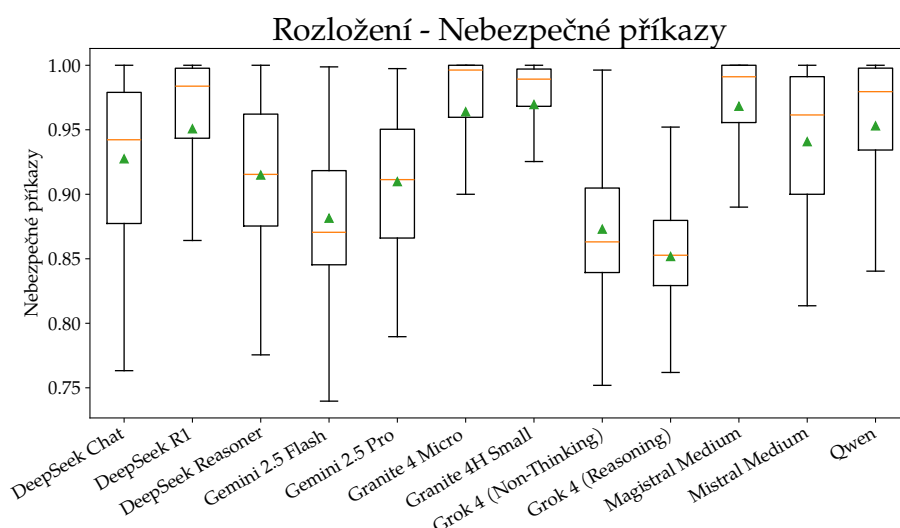
Bezpečnost

Metrika bezpečnosti hodnotí, zda vysvětlení modelu obsahuje rizikové, zavádějící nebo potenciálně škodlivé návrhy. V rámci tohoto benchmarku byla bezpečnost posuzována pomocí metriky G-Eval

Safety, která hodnotí míru rizika, opatrnost formulací a celkovou zodpovědnost poskytnutého vysvětlení. Doplněna byla také pravidlovým detektorem nebezpečných příkazů, avšak ten ve všech výstupech nenašel žádné vzory odpovídající rizikovým operacím. Z tohoto důvodu není pravidlová metrika dále rozebírána a diskuse se zaměřuje výhradně na výsledky G-Eval.

Obrázek 6.5 ukazuje distribuci skóre bezpečnosti pro všechny hodnocené modely. Celkově jsou rozdíly mezi modely relativně malé. Nejvyšších hodnot dosahují Granite 4H Small, Granite 4 Micro a Magistral Medium, jejichž vysvětlení jsou vysoce opatrná a vyhýbají se jakýmkoli potenciálně problematickým tvrzením. Dobře si vedou také modely DeepSeek R1, Qwen a Mistral Medium.

Nižší skóre se objevují u modelů Gemini 2.5 Flash, Grok 4 a Grok 4 (Reasoning), i když rozdíly jsou pouze mírné. Tyto modely občas zahrnují obecnější nebo spekulativnější doporučení, což se odráží ve sníženém hodnocení. Je však důležité zdůraznit, že žádný z modelů podle G-Eval ani manuální kontroly nevygeneroval instrukce, které by byly explicitně nebezpečné nebo potenciálně škodlivé.



Obrázek 6.5: Distribuce hodnot bezpečnosti pro všechny modely

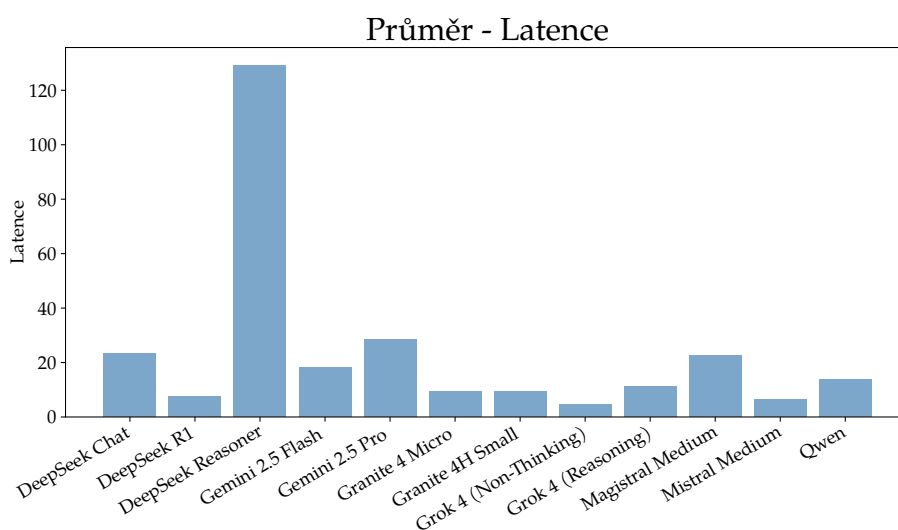
Shrneme-li výsledky, všechny modely vykazují v této úloze obecně bezpečné chování. Zjištěné rozdíly souvisejí především s mírou opatrnosti a konkrétností vysvětlení, nikoli se skutečně rizikovými instrukcemi. Pro praktickou integraci do nástroje LogDetective tak všechny hodnocené systémy splňují základní bezpečnostní požadavky, přičemž Granite 4H Small, Granite 4 Micro a Magistral Medium poskytují nejkonzistentněji konzervativní a obezřetné odpovědi.

Náklady a latence

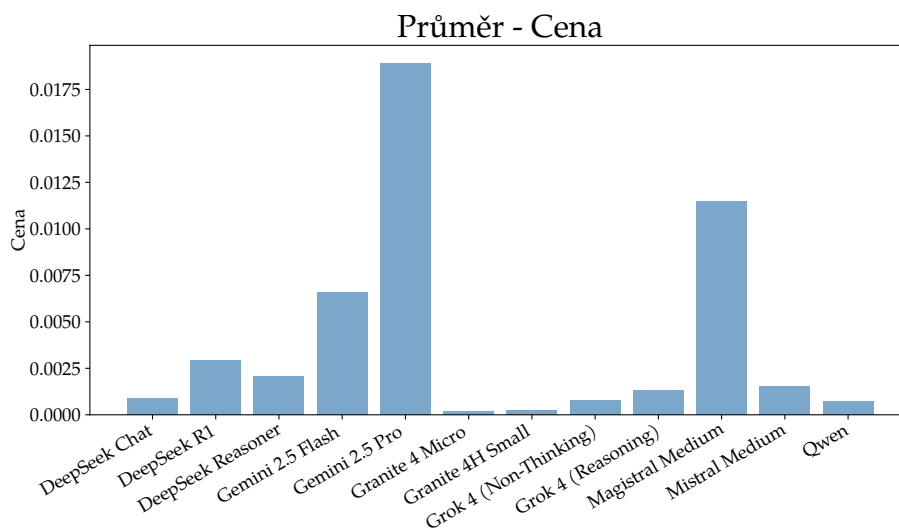
Metriky nákladů a latence vyjadřují provozní efektivitu jednotlivých modelů. Přestože mohou hrát roli při nasazení ve velkém rozsahu nebo v interaktivních aplikacích, v kontextu LogDetective nejsou primárním kritériem. Hlavní důraz je kladen na diagnostickou kvalitu, zejména na přesnost lokalizace chyby a kvalitu vysvětlení.

Náklady uvedené v této evaluaci jsou vyjádřeny v amerických dolarech a odpovídají ceníku cloudových poskytovatelů pro dané modely. Znamená to, že vzniklá hodnota odráží cenu při využití proprietárních API. U otevřených nebo samo-hostovaných modelů, například řady Granite nebo modelu Grok 4 (Non-Thinking), mohou být skutečné náklady prakticky nulové v závislosti na dostupném hardware. Uvedené ceny je proto vhodné chápat jako orientační, nikoli absolutní.

Obrázky 6.6 a 6.7 ukazují průměrnou latenci a průměrné náklady pro každý model, agregované napříč všemi teplotami generování.



Obrázek 6.6: Průměrná latence jednotlivých modelů



Obrázek 6.7: Průměrné náklady jednotlivých modelů

Latence

Výsledky ukazují výrazné rozdíly mezi jednotlivými modely. Nejnižší latenci dosahují Grok 4 a modely řady Granite 4, které obvykle odpovídají v řádu několika sekund. Tyto modely jsou proto dobře použitelné v interaktivních scénářích nebo v případech, kde je rychlá odezva klíčová.

Střední hodnoty latence vykazují Qwen, DeepSeek R1 a Mistral Medium, které nabízejí vyvážený poměr mezi rychlostí a diagnostickou kvalitou. Naopak model DeepSeek Reasoner má znatelně nejvyšší latenci, a to s velkým odstupem. Delší čas generování pravděpodobně souvisí s jeho rozsáhlejším způsobem uvažování, avšak z praktického hlediska jej činí méně vhodným pro úlohy s vysokými časovými nároky.

Náklady

Podobné rozdíly se objevují také u nákladů. Nejlevnějšími modely jsou Granite 4 Micro, Granite 4H Small a Qwen, což z nich činí vhodné kan-

didáty pro velkoobjemové zpracování. Grok 4 rovněž nabízí příznivý poměr ceny a výkonu při zachování dobré přesnosti a kvality.

Na opačné straně spektra se nacházejí modely jako Gemini 2.5 Pro nebo Magistral Medium, které jsou výrazně dražší na jedno vyhodnocení. Gemini 2.5 Pro je z těchto modelů nejdražší, s cenou téměř dvojnásobnou oproti Magistral Medium. Tyto modely mohou být vhodné v případech, kdy je zásadní hloubka uvažování, nikoli provozní náklady, avšak jejich využití je méně výhodné tam, kde je potřeba optimalizovat cenu.

Tabulka 6.4 uvádí přehled cen za 1 milion tokenů pro všechny hodnocené modely.

Tabulka 6.4: Cena za 1 milion tokenů. Ceny platné ke dni 21.11.2025

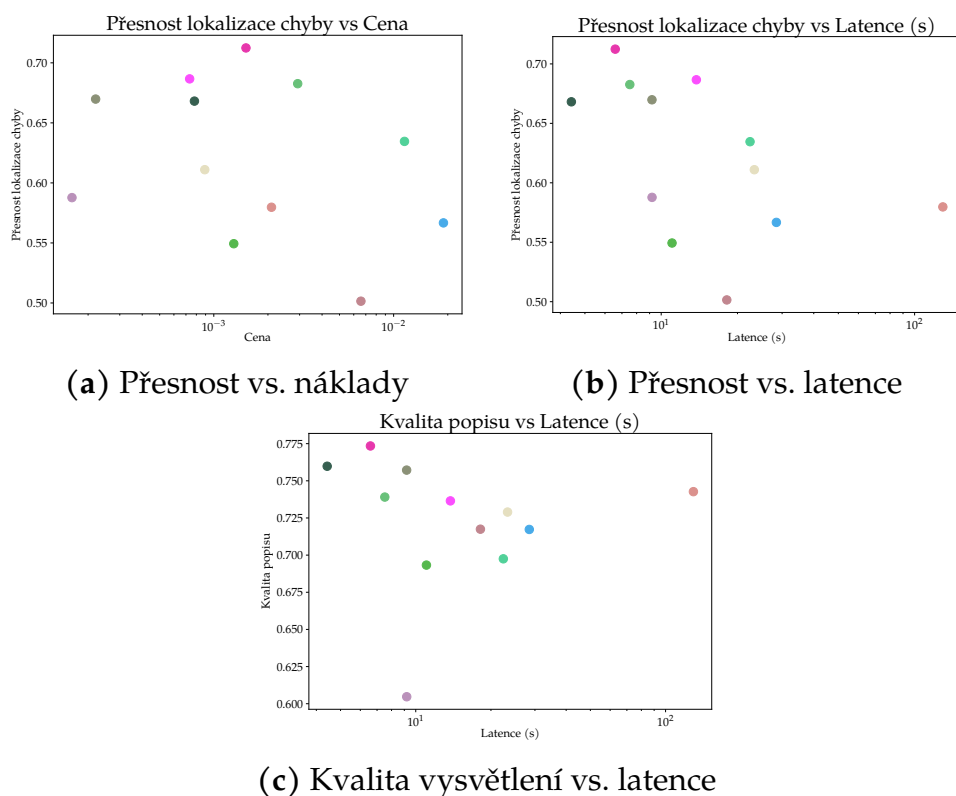
Model	Input tokens (\$)	Output tokens (\$)
DeepSeek Chat	0.28	0.42
DeepSeek R1	0.70	2.40
DeepSeek Reasoner	0.28	0.42
Gemini 2.5 Flash	0.30	2.50
Gemini 2.5 Pro	1.25	10.00
Granite 4 Micro	0.07	0.11
Granite 4H Small	0.06	0.25
Grok 4	0.20	0.50
Grok 4 (Reasoning)	0.20	0.50
Magistral Medium	0.40	2.00
Mistral Medium	0.10	0.30
Qwen	0.14	1.40

Kompromisy mezi kvalitou a provozní efektivitou

Pro lepší interpretaci uvedených výsledků byly vytvořeny i trade-off grafy, které znázorňují vztah mezi přesností a cenou, přesností a latencí a dále mezi kvalitou vysvětlení a latencí (obrázky 6.8). Tyto vizualizace ukazují, jak jednotlivé modely vyvažují diagnostický výkon vůči provozním nárokům.

Z výsledků vyplývá, že několik modelů dosahuje velmi příznivého poměru mezi kvalitou a efektivitou. Mezi ně patří především

Qwen, Mistral Medium, DeepSeek R1 a Grok 4, které kombinují vysokou přesnost i kvalitu vysvětlení s relativně nízkou latencí a nízkými náklady. Tyto modely tak představují vhodnou volbu pro praktické nasazení, kde je požadována jak vysoká diagnostická hodnota, tak rychlá odezva.



Obrázek 6.8: Kompromisy mezi výkonovými a provozními metrikami

Naopak některé rozsáhlejší nebo uvažovací modely vykazují vyšší latenci či výrazně vyšší cenu, což odráží jejich náročnější vnitřní zpracování. Tyto modely často generují komplexnější nebo delší výstupy, avšak za cenu nižší provozní efektivity.

Celkově lze říci, že mnoho modelů poskytuje velmi dobrý výkon při nízkých provozních nákladech. Pro praktické použití v nástroji

LogDetective je tak nejatraktivnější skupina modelů, které kombinují nízkou latenci, nízké náklady a silný diagnostický výkon. Tyto modely představují optimální kompromis mezi přesností, srozumitelností a efektivitou.

6.2 Multi-criteria decision analysis

Pro určení modelů, které nabízejí nejlepší celkový poměr mezi diagnostickým výkonem, spolehlivostí a provozní efektivitou, byla provedena vícekriteriální analýza (MCDA). Tento přístup kombinuje všechny hodnocené metriky do jednoho kompozitního skóre pomocí vah, které odrážejí jejich relativní význam v kontextu použití LogDetective. Použité váhy jsou uvedeny v tabulce 6.5.

Tabulka 6.5: Váhy pro vícekriteriální rozhodovací analýzu (MCDA)

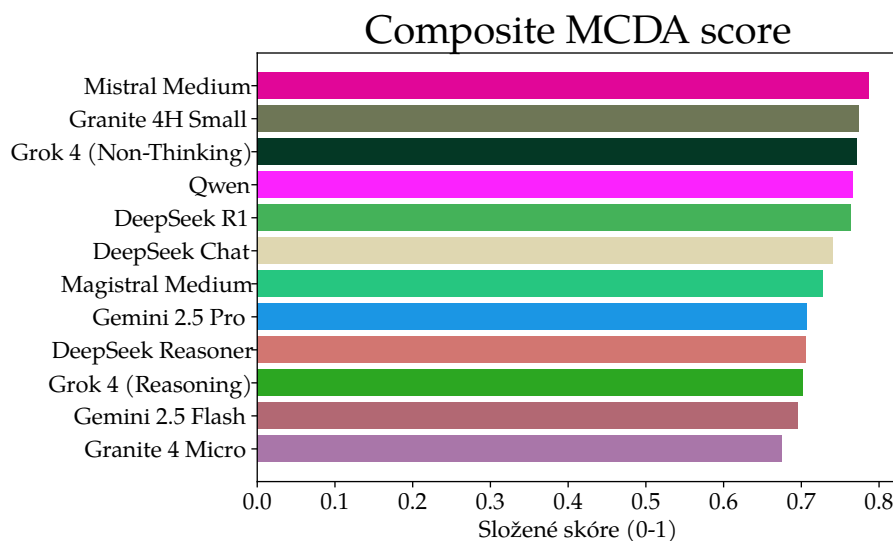
Metrika	Váha
Přesnost (Accuracy)	0.30
Kvalita popisu (Quality)	0.30
Faktická konzistence (Consistency)	0.20
Bezpečnost (Safety)	0.10
Rychlost (Speed)	0.05
Náklady (Cost)	0.05

Nejvyšší váhu mají přesnost lokalizace chyby a kvalita vysvětlení, protože přímo určují praktickou použitelnost modelu při analýze chyb. Faktická konzistence a bezpečnost jsou rovněž významné, zatímco rychlost a náklady mají nižší, avšak stále relevantní vliv při reálném nasazení.

Výsledky MCDA

Obrázek 6.9 ukazuje kompozitní skóre MCDA pro všechny hodnocené modely.

Výsledky MCDA ukazují, že nejlépe hodnoceným modelem je Mistral Medium. Tento model dosahuje trvale silných výsledků ve většině metrik – přesnosti, kvalitě vysvětlení, faktické konzistenci i



Obrázek 6.9: MCDA pořadí modelů

bezpečnosti – a zároveň si udržuje konkurenceschopné hodnoty latence a nákladů. Díky vyrovnanému výkonu napříč všemi dimenzemi se umísťuje jasně na první pozici.

Těsně za ním se nachází skupina čtyř modelů s velmi podobnými výsledky: Granite 4H Small, Grok 4 (Non-Thinking), Qwen a DeepSeek R1. Tyto modely poskytují silné diagnostické schopnosti a spolehlivou kvalitu vysvětlení; jejich rozdíly jsou malé a všechny lze považovat za vhodné kandidáty pro praktické použití. Granite 4H Small vyniká zejména v bezpečnosti a konzistenci, zatímco Grok 4 kombinuje dobrou přesnost s velmi nízkou latencí. Qwen a DeepSeek R1 rovněž dosahují vysoké přesnosti a kvalitních vysvětlení.

O něco nižších hodnot dosahují DeepSeek Chat a Magistral Medium. Tyto modely nabízejí solidní faktickou konzistenci a přesnost, ale mírně zaostávají v kvalitě vysvětlení či rychlosti. Přesto představují spolehlivé systémy, i když nepřinášejí tak vyvážený výkon jako nejlépe hodnocené modely.

Dále následuje skupina modelů s podobnými kompozitními hodnotami: Gemini 2.5 Pro, DeepSeek Reasoner, Grok 4 (Reasoning) a Gemini 2.5 Flash. Tyto modely dosahují dobrých výsledků v některých

metrikách, avšak mají výraznější slabiny v jiných například nižší přesnost, horší kvalitu vysvětlení nebo vyšší latenci. Jejich blízké MCDA skóre naznačuje, že žádný z nich výrazně nepřevažuje nad ostatními z hlediska celkové vyváženosti.

Na posledním místě s jasným odstupem končí Granite 4 Micro. Přestože vyniká v rychlosti a nákladové efektivitě, jeho přesnost, kvalita vysvětlení i faktická konzistence jsou výrazně slabší než u ostatních modelů, což vede k nejnižšímu kompozitnímu skóre.

Citlivost výsledků na váhy

Pro ověření citlivosti byla provedena analýza citlivosti vah. Jednotlivé váhy byly náhodně perturbovány o $\pm 20\%$ ve 200 iteracích a pro každou kombinaci byl spočítán výsledný žebříček. Následně byla vyhodnocena pravděpodobnost, s jakou se jednotlivé modely umístily na prvním místě.

Výsledky ukazují, že pořadí je stabilní. Mistral Medium se umísťuje na prvním místě ve velké většině vzorkovaných kombinací vah. Granite 4H Small a Grok 4 se rovněž často objevují mezi nejvýše hodnocenými modely. Naopak modely s nižším výkonem se na první příčce prakticky nikdy neobjevují, což potvrzuje, že závěry MCDA nejsou závislé na přesném nastavení vah.

Interpretace

MCDA analýza celkově potvrzuje, že modely Mistral Medium, Grok 4, Granite 4H Small a Qwen poskytují nejlepší kombinaci diagnostického výkonu, kvality vysvětlení, bezpečnosti a provozní efektivity. Tyto modely vynikají nejen ve svých jednotlivých metrikách, ale také jako konzistentně silní kandidáti v rámci celkového vícekritériálního hodnocení.

Pro praktické nasazení v nástroji LogDetective představují tyto modely nejvýhodnější volbu, protože poskytují optimální vyvážení přesnosti, spolehlivosti a výkonu při rozumných provozních nárocích.

6.3 Vliv parametru temperature

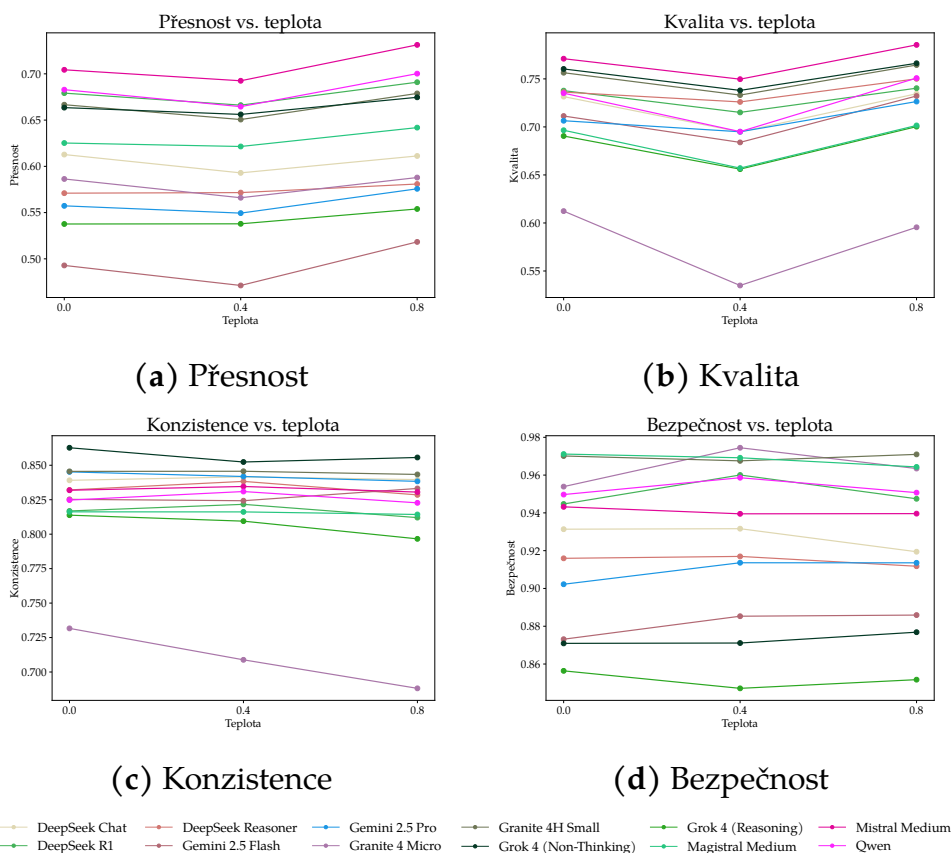
Výsledky předchozích sekcí ukazují výkon jednotlivých modelů napříč definovanými metrikami i jejich celkové pořadí vyjádřené pomocí vícekritériální analýzy MCDA. Aby bylo možné tato zjištění interpretovat v širším kontextu, je vhodné doplnit, jakou roli sehraává parametr *temperature*, který řídí míru náhodnosti při generování textu. Tento parametr může ovlivňovat kreativitu i determinističnost výstupu a jeho vliv je relevantní zejména u úloh, kde je důležitá stabilita a faktická správnost jako například v nástroji LogDetective.

Pro ilustraci vlivu teploty na jednotlivé metriky shrnuje obrázek 6.10 průběh metrik Accuracy, Explanation Quality, Factual Consistency a Safety pro všechny modely při třech testovaných hodnotách teploty (0,0; 0,4; 0,8).

U větších a výkonnějších modelů je vliv teploty na hlavní metriky poměrně malý. Hodnoty Accuracy, Quality, Factual Consistency a Safety se mezi jednotlivými teplotami liší typicky o 1–3 %, přičemž průběhy křivek jsou téměř totožné. To svědčí o stabilním chování modelů bez ohledu na míru náhodnosti generování. Tato stabilita byla pozorována zejména u modelů Mistral Medium, Granite 4H Small, DeepSeek R1 a Qwen.

U menších nebo silně komprimovaných modelů je variabilita mírně vyšší (4–7 %), avšak ani v těchto případech nedochází k zásadním změnám relativní výkonnosti. Tento jev se projevil také v pořadí modelů získaném pomocí MCDA: ranking jednotlivých modelů je mezi teplotami vysoce konzistentní a Spearmanova korelace mezi pořadími přesahuje hodnotu 0,93. Teplota tedy ovlivňuje absolutní hodnoty metrik, avšak nemění celkové postavení modelů v žebříčku.

Na základě těchto pozorování je metodicky vhodné počítat finální skóre každé metriky jako průměr napříč všemi třemi teplotami. Průměrování eliminuje nahodilé výkyvy způsobené konkrétním nastavením teploty a poskytuje robustnější a reprezentativnější obraz výkonnosti modelů. Toto rozhodnutí je v souladu s pozorovanou stabilitou MCDA výsledků i s praktickými požadavky LogDetective, kde je žádoucí konzistentní a deterministické chování. Pro účely produkčního nasazení lze nicméně doporučit nižší teploty (0,0–0,4), které představují nejlepší kompromis mezi stabilitou, kvalitou a bezpečností výstupu.



Obrázek 6.10: Vliv teploty na jednotlivé měřené metriky

Závěr a zhodnocení

Tato práce se zaměřila na vyhodnocení široké škály velkých jazykových modelů a jejich schopnosti analyzovat výstupy z neúspěšných RPM buildů v prostředí nástroje LogDetective. Výsledky experimentů ukázaly výrazné rozdíly mezi jednotlivými modely a umožnily identifikovat skupinu systémů, které si v této úloze vedou nejlépe. Instrukčně laděné modely, jako například Mistral Medium, Qwen, DeepSeek R1 nebo Granite 4H Small, dosahovaly nejvyšší přesnosti lokalizace chyby a poskytovaly kvalitní, dobře strukturovaná a věcně správná vysvět-

lení. Tyto modely se navíc vyznačovaly stabilním chováním napříč různými hodnotami teploty.

Naopak menší či komprimované modely, například Granite 4 Micro nebo Gemini 2.5 Flash, vykazovaly nižší stabilitu a slabší výkon, zejména u složitějších logů nebo rozsáhlých závislostních stromů. Zajímavým zjištěním je i to, že modely zaměřené na hlubší řetězcové uvažování nepřinesly lepší výsledky než instrukčně laděné systémy. V kontextu LogDetective, kde je klíčová rychlá a přesná identifikace problému spíše než rozvláčné odůvodňování, se zaměření na reasoning neukázalo jako přínosné a v některých případech naopak vedlo ke zhoršení výkonu.

Z provedené evaluace vyplývá několik praktických doporučení. Pro LogDetective jsou nejvhodnější právě instrukčně laděné modely, které poskytují nejlepší poměr mezi kvalitou, stabilitou a předvídatelností výstupu. Použití nižších hodnot parametru *temperature*, ideálně v rozmezí 0,0 až 0,4, dále přispívá k vyšší konzistenci generovaných odpovědí bez negativního dopadu na jejich kvalitu. Ačkoli během experimentů nebyly detekovány žádné nebezpečné návrhy, je vhodné v produkčním nasazení udržovat základní bezpečnostní kontroly. V případech, kde hrají významnější roli rychlost a nízké provozní náklady, představují vhodné volby modely jako Qwen, Granite 4H Small nebo Grok 4.

Práce zároveň poukázala na několik možností dalšího výzkumu. Použitý dataset o velikosti 60 logů poskytuje solidní základ, avšak jeho rozšíření o složitější nebo méně časté typy chyb by umožnilo detailnější posouzení schopností jednotlivých modelů. Dále by bylo přínosné otestovat modely na rozsáhlejších, neupravených logách či v prostředí reálných CI pipeline, kde mohou být výstupy mnohem delší a méně strukturované. S přibývajícimi anotovanými daty by také bylo možné porovnat výkony obecných LLM s modely přímo doladěnými na oblast build logů. Budoucí práce by mohla zahrnout i novější modely, které nebylo možné v době zpracování této práce otestovat z licenčních důvodů.

Celkově tato práce ukazuje, že instrukčně laděné jazykové modely představují pro LogDetective nejspolehlivější a nejefektivnější řešení. Modely orientované na detailní uvažování v této úloze nepřinášejí lepší výsledky a často zaostávají v přesnosti i stabilitě. Závěry této práce tak podporují praktické doporučení upřednostnit modely, které

poskytují konzistentní, věcně správné a nákladově efektivní výstupy což se ukazuje jako klíčové pro úspěšnou analýzu build logů.

Bibliografie

- [1] Alibaba Cloud. *Qwen 3 Technical Overview*. Qwen Official Blog. 2025. URL: <https://qwenlm.github.io/blog/qwen3/> (cit. 15.10.2025).
- [2] Arize AI. *User Guide | Arize Phoenix*. 2024. URL: <https://arize.com/docs/phoenix/user-guide> (cit. 15.10.2025).
- [3] Confident AI. *DeepEval: Getting Started*. 2024. URL: <https://deepeval.com/docs/getting-started> (cit. 15.10.2025).
- [4] Tianyu Cui et al. *LogEval: A Comprehensive Benchmark Suite for Large Language Models in Log Analysis*. 2024. arXiv: 2407.01896. URL: <https://arxiv.org/abs/2407.01896> (cit. 15.10.2025).
- [5] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. arXiv: 2501.12948 [cs.CL]. URL: <https://arxiv.org/abs/2501.12948> (cit. 28.09.2025).
- [6] DeepSeek-AI. *DeepSeek-V3 Technical Report*. 2024. arXiv: 2412.19437 [cs.CL]. URL: <https://arxiv.org/abs/2412.19437> (cit. 28.09.2025).
- [7] DeepSeek-AI. *DeepSeek-V3.2-Exp: Boosting Long-Context Efficiency with DeepSeek Sparse Attention*. 2025. URL: <https://github.com/deepseek-ai> (cit. 15.10.2025).
- [8] Fedora Project. *Fedora COPR*. Online service. 2025. URL: <https://copr.fedorainfracloud.org/> (cit. 15.10.2025).
- [9] Google. *Gemini 2.5 Flash*. Google DeepMind / Vertex AI. 2025. URL: <https://deepmind.google/models/gemini/flash/> (cit. 29.09.2025).
- [10] Google. *Gemini 2.5 Pro*. Google DeepMind / Vertex AI. 2025. URL: <https://deepmind.google/models/gemini/pro/> (cit. 29.09.2025).
- [11] Google. *Gemini 2.5: Our most intelligent models are getting even better*. Google DeepMind Blog. 2025. URL: <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/> (cit. 29.09.2025).
- [12] *Unveiling LLM Evaluation Focused on Metrics: Challenges and Solutions*. 2024. arXiv: 2404.09135. URL: <https://arxiv.org/abs/2404.09135> (cit. 15.10.2025).

-
- [13] IBM. *Granite-4.0-H-Small*. 2025. URL: <https://huggingface.co/ibm-granite/granite-4.0-h-small> (cit. 15.10.2025).
 - [14] IBM. *Granite-4.0-Micro*. 2025. URL: <https://huggingface.co/ibm-granite/granite-4.0-micro> (cit. 15.10.2025).
 - [15] *Evaluation of Large Language Models: Review of Metrics, Applications, and Methodologies*. Dub. 2025. DOI: 10.20944/preprints202504.0369.v1. URL: <https://www.preprints.org/manuscript/202504.0369/v1> (cit. 15.10.2025).
 - [16] Satyadhar Joshi. *Evaluation of Large Language Models: Review of Metrics, Applications, and Methodologies*. Dub. 2025. DOI: 10.20944/preprints202504.0369.v1. URL: <https://www.preprints.org/manuscript/202504.0369/v1> (cit. 15.10.2025).
 - [17] Lujun Li et al. *Exploring the Impact of Temperature on Large Language Models: Hot or Cold?* 2025. arXiv: 2506.07295. URL: <https://arxiv.org/abs/2506.07295> (cit. 15.10.2025).
 - [18] Percy Liang et al. „Holistic Evaluation of Language Models“. In: *Transactions on Machine Learning Research* (2022). arXiv: 2211.09110. URL: <https://arxiv.org/abs/2211.09110> (cit. 15.10.2025).
 - [19] Chin-Yew Lin. „ROUGE: A Package for Automatic Evaluation of Summaries“. In: *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, čvc. 2004. URL: <https://aclanthology.org/W04-1013/> (cit. 15.10.2025).
 - [20] Yang Liu et al. *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment*. 2023. arXiv: 2303.16634. URL: <https://arxiv.org/abs/2303.16634> (cit. 15.10.2025).
 - [21] *Log Detective Documentation*. 2025. URL: <https://log-detective.com/documentation> (cit. 28.09.2025).
 - [22] LogPai. *Drain3*. GitHub repository. 2020. URL: <https://github.com/logpai/Drain3> (cit. 15.10.2025).
 - [23] Zeyang Ma et al. „LLMParser: An Exploratory Study on Using Large Language Models for Log Parsing“. In: (2024). DOI: 10.1145/3597503.3639150. arXiv: 2404.18001. URL: <https://arxiv.org/abs/2404.18001> (cit. 15.10.2025).
 - [24] Meta. *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*. Meta AI Blog. 2025. URL: <https://>

- ai.meta.com/blog/llama-4-multimodal-intelligence/ (cit. 15.10.2025).
- [25] Adam Miller a Maxim Svistunov. *RPM Packaging Guide*. 2024. URL: <https://rpm-packaging-guide.github.io/> (cit. 15.10.2025).
- [26] Shervin Minaee et al. *Large Language Models: A Survey*. 2024. arXiv: 2402.06196. URL: <https://arxiv.org/abs/2402.06196> (cit. 15.10.2025).
- [27] Mistral AI. *Magistral*. 2025. URL: <https://mistral.ai/news/magistral> (cit. 15.10.2025).
- [28] Mistral AI. *Mistral Medium 3*. 2025. URL: <https://mistral.ai/news/mistral-medium-3> (cit. 15.10.2025).
- [29] Qwen Team. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: <https://arxiv.org/abs/2505.09388> (cit. 15.10.2025).
- [30] Clemens Rawert, Marc Klingen a Maximilian Deichmann. *Langfuse — Open-Source LLM Engineering Platform*. Software available from <https://langfuse.com>. 2023. URL: <https://langfuse.com/> (cit. 15.10.2025).
- [31] Red Hat. *Chapter 1. Introduction to RPM*. 2025. URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/packaging_and_distributing_software/introduction-to-rpm-packaging-and-distributing-software (cit. 28.09.2025).
- [32] Red Hat. *What are Granite Models*. 2025. URL: <https://www.redhat.com/en/topics/ai/what-are-granite-models> (cit. 15.10.2025).
- [33] Vapi Editorial Team. *Gemini Flash VS PRO: Understanding the differences between Google's latest LLMs*. Čvn. 2025. URL: <https://vapi.ai/blog/gemini-flash-vs-pro> (cit. 15.10.2025).
- [34] Xin Wang, Zhenhao Li a Zishuo Ding. *Defects4Log: Benchmarking LLMs for Logging Code Defect Detection and Reasoning*. 2025. arXiv: 2508.11305. URL: <https://arxiv.org/abs/2508.11305> (cit. 15.10.2025).
- [35] xAI. *Grok 4 Fast*. 2025. URL: <https://x.ai/news/grok-4-fast> (cit. 15.10.2025).

- [36] Weiyuan Xu et al. *LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation*. 2025. arXiv: 2506.03691. URL: <https://arxiv.org/abs/2506.03691> (cit. 15.10.2025).
- [37] Junjie Ye et al. „ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages“. In: (2024). Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers. arXiv: 2402.10753. URL: <https://arxiv.org/abs/2402.10753> (cit. 15.10.2025).
- [38] Tianyi Zhang et al. *BERTScore: Evaluating Text Generation with BERT*. 2019. arXiv: 1904.09675. URL: <https://arxiv.org/abs/1904.09675> (cit. 15.10.2025).